

# 芯片安全方案

## NGU800P安全方案

### 1. 文档概述

#### 1.1 文档范围

基于NGU800P芯片需求，集成OSR eHSM IP，满足芯片基础安全能力及AI数据中心场景，设计NGU800P安全架构与实施方案。本文档描述NGU800芯片边界内的安全需求及die外MCU安全本文档内容包括:安全子系统功能，多Die场景安全，DICE/SPDM方案等

#### 1.2 安全目标

NGU800P安全体系目标如下：

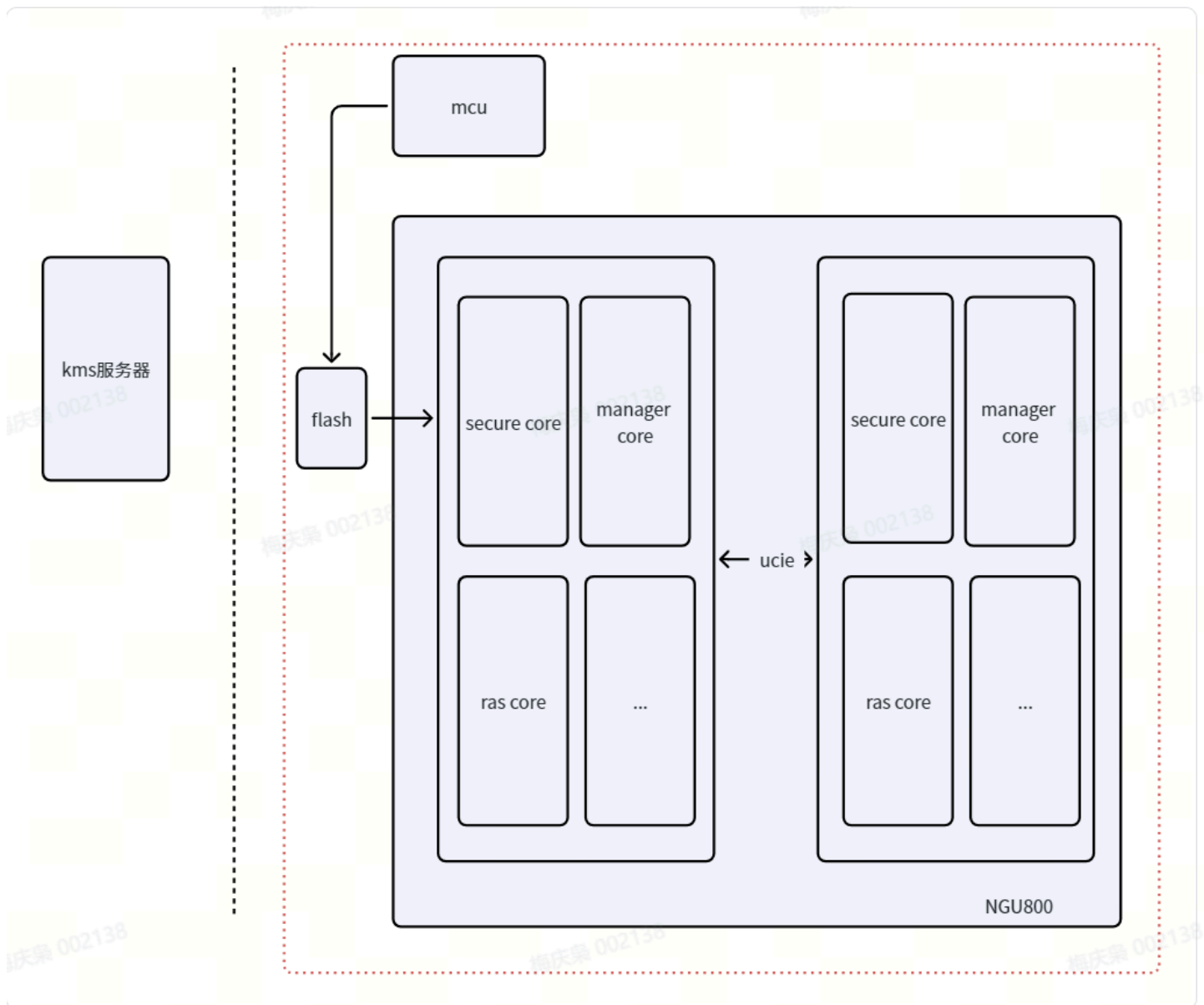
| 安全目标              | 描述                           |
|-------------------|------------------------------|
| Root of Trust     | 提供硬件信任根eHSM                  |
| Secure Boot       | 确保eHSM FW->c908->其他关键固件启动链可信 |
| Key Protection    | 保护根密钥及业务密钥                   |
| Firmware Security | 防止固件篡改与回滚                    |
| DICE/Attestation  | 支持远程证明/度量                    |
| Data Protection   | 支持敏感数据保护                     |
| Lifecycle Control | 生命周期安全管理                     |
| Debug Protection  | 防止非法调试                       |
| Compliance        | 支持国密一级认证能力                   |
| die 间安全           | 保证die间启动镜像安全                 |

### 4. 方案架构

#### 4.1 SOC 需求列表

目 芯片安全需求

#### 4.2 安全边界



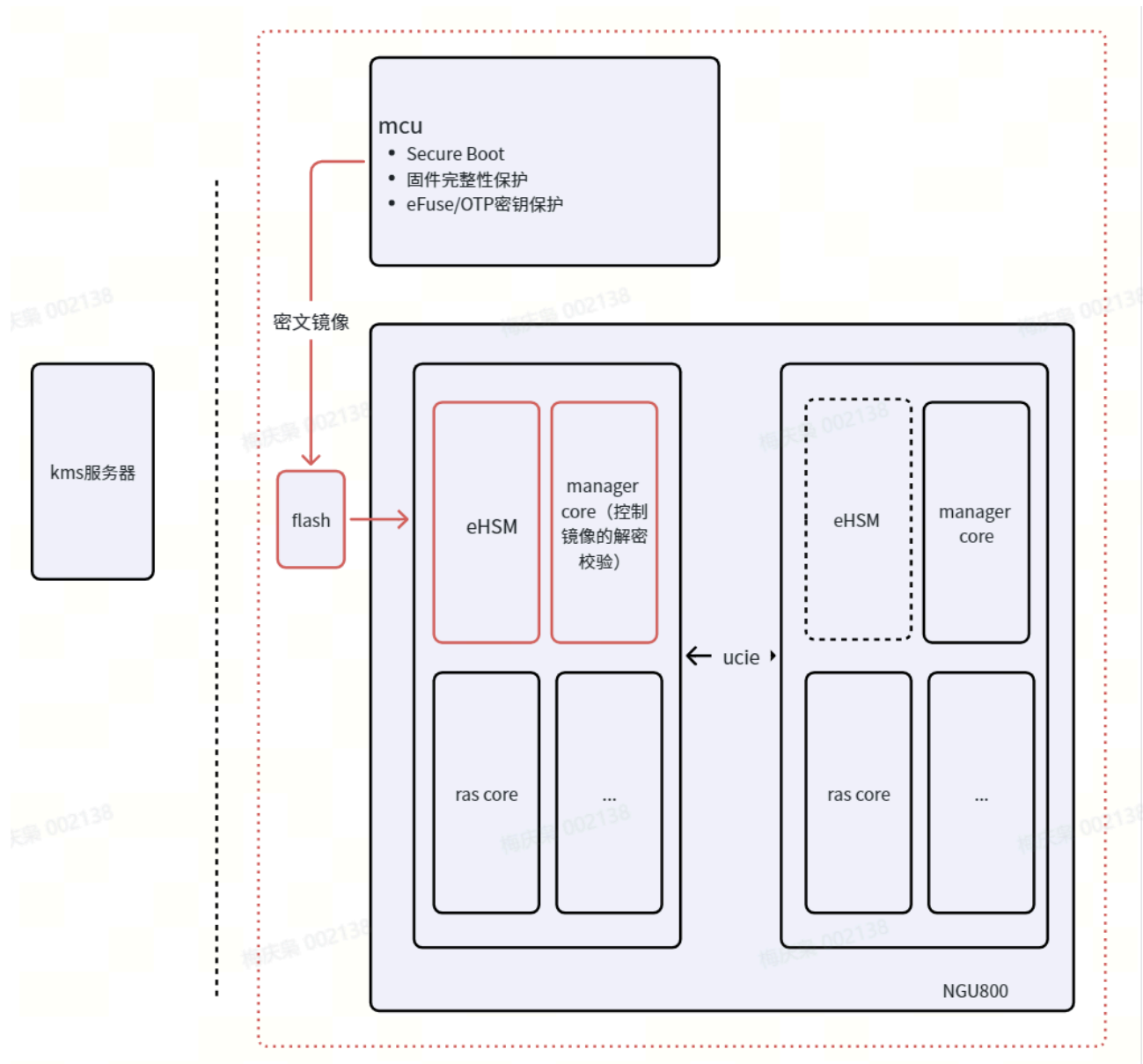
芯片的安全边界包括NGU800和die 外MCU。

BMC 不在芯片的安全边界内。

## 5. die外MCU

### 📖 die 外MCU选项安全分析

- die外mcu为外购芯片。
- die外mcu的固件安全，启动安全由mcu自己提供。
- mcu在烧写flash前会对自身固件进行校验（独立于ngu800之外的签名体系）
- mcu的安全启动不可绕过，无其他旁路启动路径。
- mcu搬运镜像到flash为密文搬运，传输路径上没有明文，验签和解密由NGU800的eHSM实现。



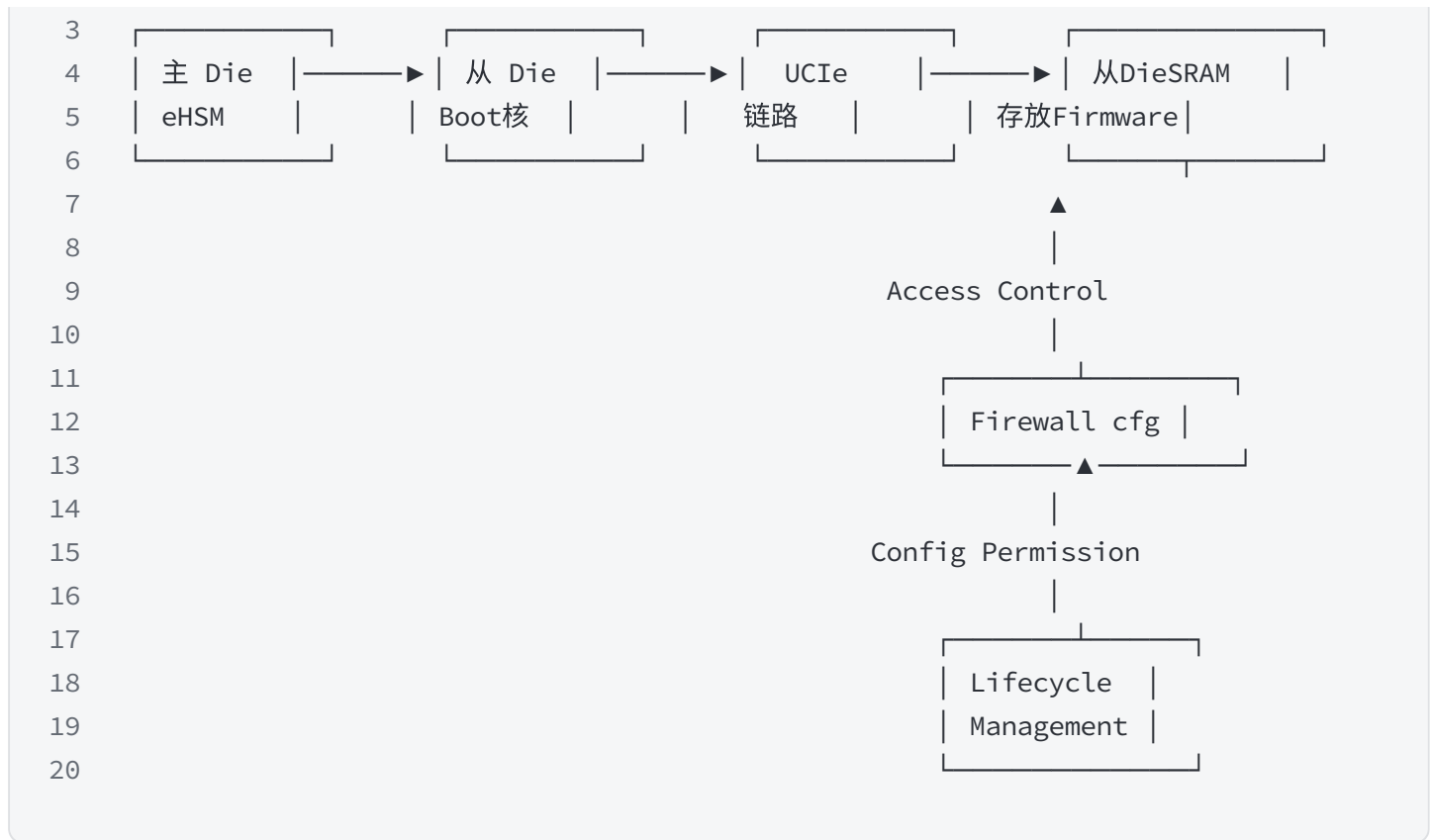
## 6. 多Die安全方案

当前启动方案：

- 主die执行安全启动，主Die eHSM 执行从die镜像解密验签
- 从Die使用主Die验签解密后的镜像进行启动
- 从die镜像在主die解密验签后通过UCIe传输，UCIe点对点从主die将镜像传输到从die的sram
- 支持Firewall
- Firewall 限制master访问从die的sram权限，固件在sram里的存储区域只有对应的核能访问
- 生命周期管控firewall的配置权限，USER阶段firewall修改配置需要鉴权

代码块

```
1 ===== 镜像数据流 =====>
2
```



安全服务：

Die 0 ：ehsm提供完整的安全服务

Die 1：启动镜像由Die 0执行验签解密

固件的测量，远程证明由die0 执行

die 1 的LCS权限管控由die 0实现

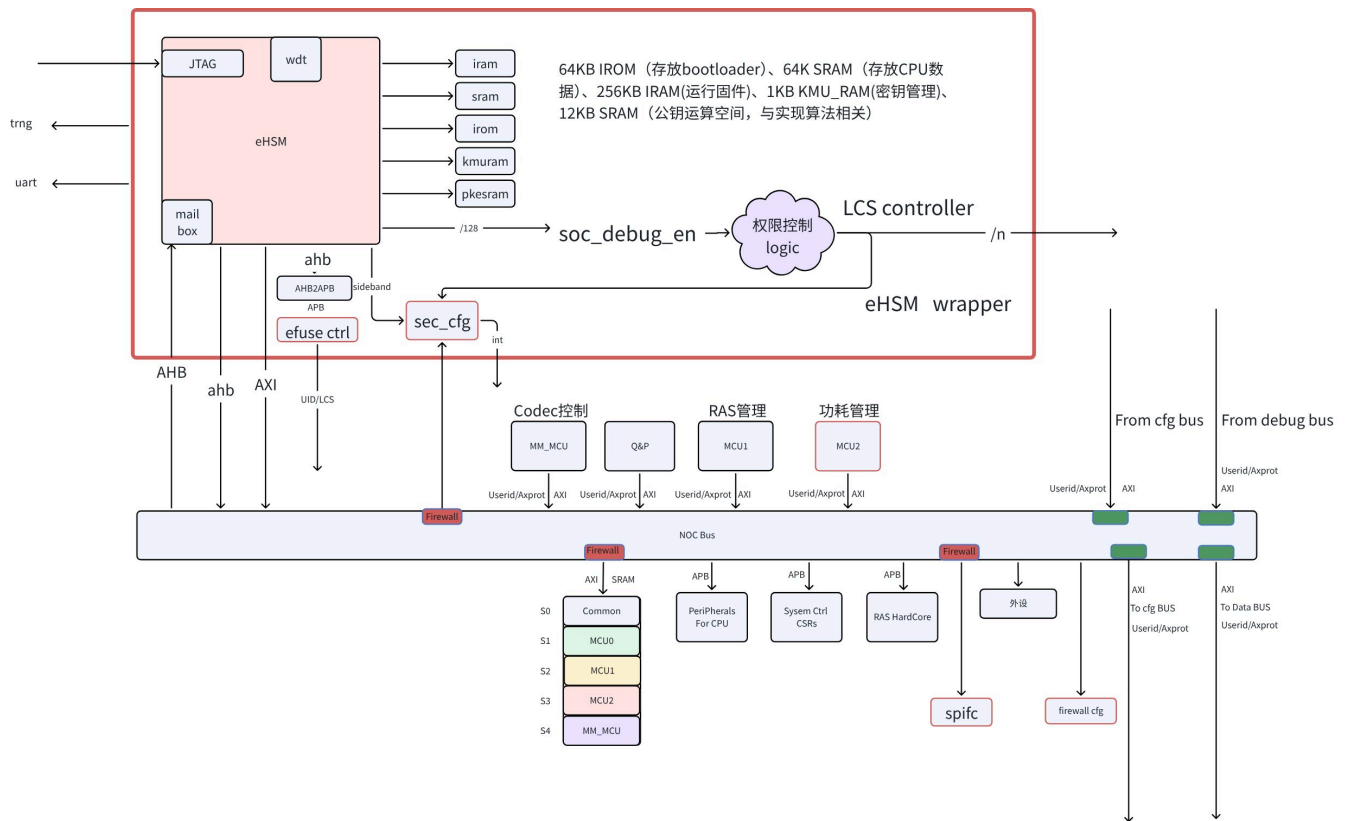
die 0 Firewall 由die 0 功耗C908配置，die 1 firewall 由die 1 功耗C908配置

die 0 Firewall 由die 0 启动C908配置，die 1 firewall 由die 1 启动C908配置

## 7. 总体安全方案

### 7.1 安全管理子系统示意图

eHSM的位置示意图如下：



eHSM位于管理子系统内，eHSM的交互组件如下图所示：

- 基于启动核C908 Risc-v CPU+OSR eHSM IP完成安全子系统功能
- eHSM相对封闭，其它Master无法直接访问，外部系统只能通过mailbox建立和eHSM的业务联系
- 管理子系统的启动C908和eHSM在系统中权限最高，可通过config bus和data bus访问系统中的其它空间
- sec\_cfg为安全专用寄存器，sec\_cfg提供安全的error、information记录，及其他一些寄存器服务。
- OSR eHSM 提供基础的安全硬件信任根
- eHSM可通过总线访安全efuse，安全efuse在总线上只有ehsm有访问通路，efuse中保存eHSM需要用到的关键信息
- eHSM可访问SOC的memory 空间，需参考memory划分表。
- eHSM debug功能包括：riscv core debug，配合debug子系统进行debug。
- eHSM有自己独立的boot rom，独立封闭运行，实现各级固件的验签、解密、密钥管理、芯片生命周期管理等

**eHSM wrapper相关的的组件：**

- eHSM：安全信任根，包含一组jtag接口，包含sram接口，包括对外通信的ahb/AXI接口。

- 5组SRAM：64KB IROM（存放bootloader）、64K SRAM（存放CPU数据）、256KB IRAM(运行固件)、1KB KMU\_RAM(密钥管理)、12KB SRAM（公钥运算空间，与实现算法相关）
- sec\_cfg:寄存器模块，解析并保存error/information等信息
- efuse：安全efuse，用于存储eHSM信息、芯片配置、硬件标识等。提供autoload，读写保护功能。只能由eHSM进行访问。

### 7.1.1 C908与eHSM的交互流程

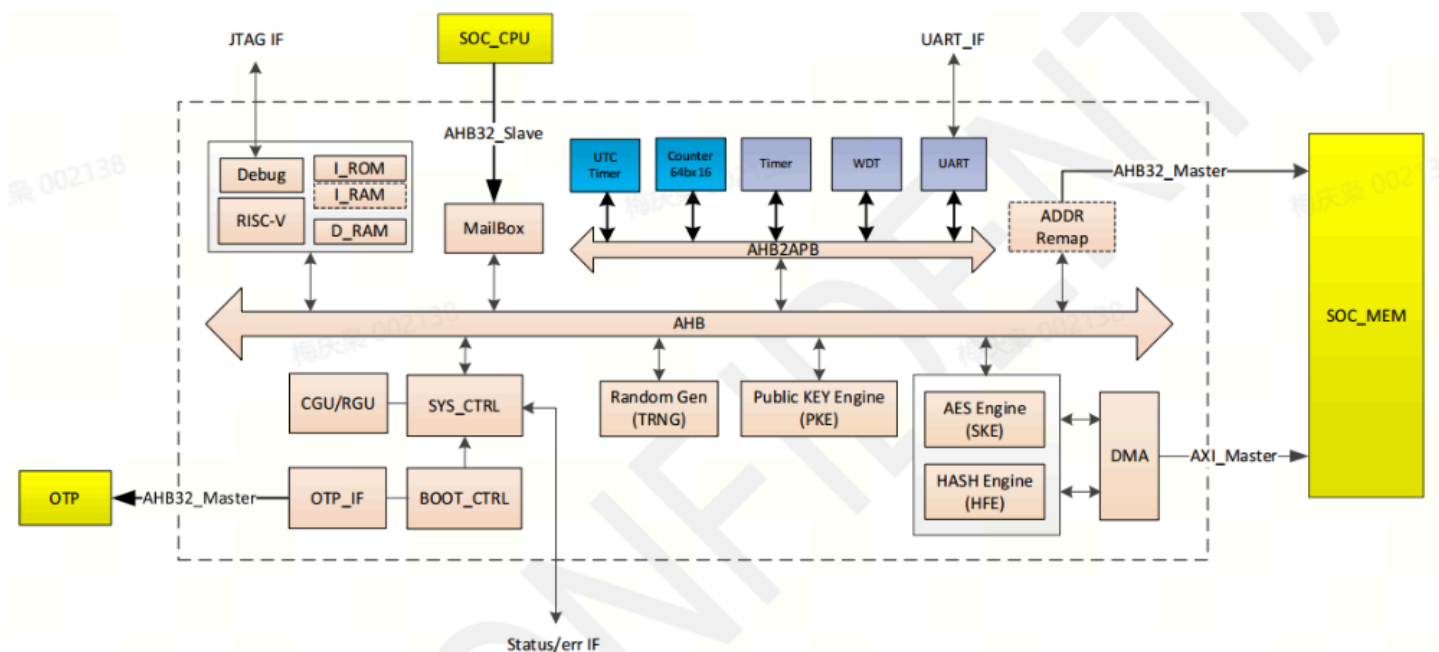
管理子系统的C908与 eHSM 之间通过 Mailbox 进行命令和响应的交互，通过 SOC\_MEM进行数据的传递。SOC 系统与 eHSM 交互过程如下：

1. C908 将待处理的数据存入 SOC\_MEM 中。
2. C908 通过 Mailbox 将待处理数据的地址、处理结果输出地址、所需操作模式(如加解密)等信息通过 command 的形式发送给 eHSM，并触发 eHSM 中断。
3. eHSM 接收并解析 command 命令，从 SOC\_MEM 取出数据并进行处理。
4. eHSM 处理完成，将结果放置在指定的 SOC\_MEM 相应地址中。
5. eHSM 发送 Response 置起 Mailbox 中的完成标识，C908通过轮询/中断方式获知eHSM 处理完成标识。
6. C908 从 SOC\_MEM 相应地址中取出结果。

一次交互过程结束

## 7.2 eHSM

eHSM 以软硬件结合的方式实现安全控制、密码运算等功能。硬件以 AMBA 总线为基础构建，包括 32 位 RISC-V CPU、安全控制系统、对称算法引擎（SKE）、公钥算法引擎（PKE）、哈希算法引擎（HASH）、真随机数发生器（TRNG）、外设单元等。同时内部部署了用于检测随机硬件失效的安全机制。eHSM结构如下图：



支持的功能：安全启动、安全调试、密钥管理、密码服务、生命周期管理、身份认证、安全升级、安全存储、证书链管理；

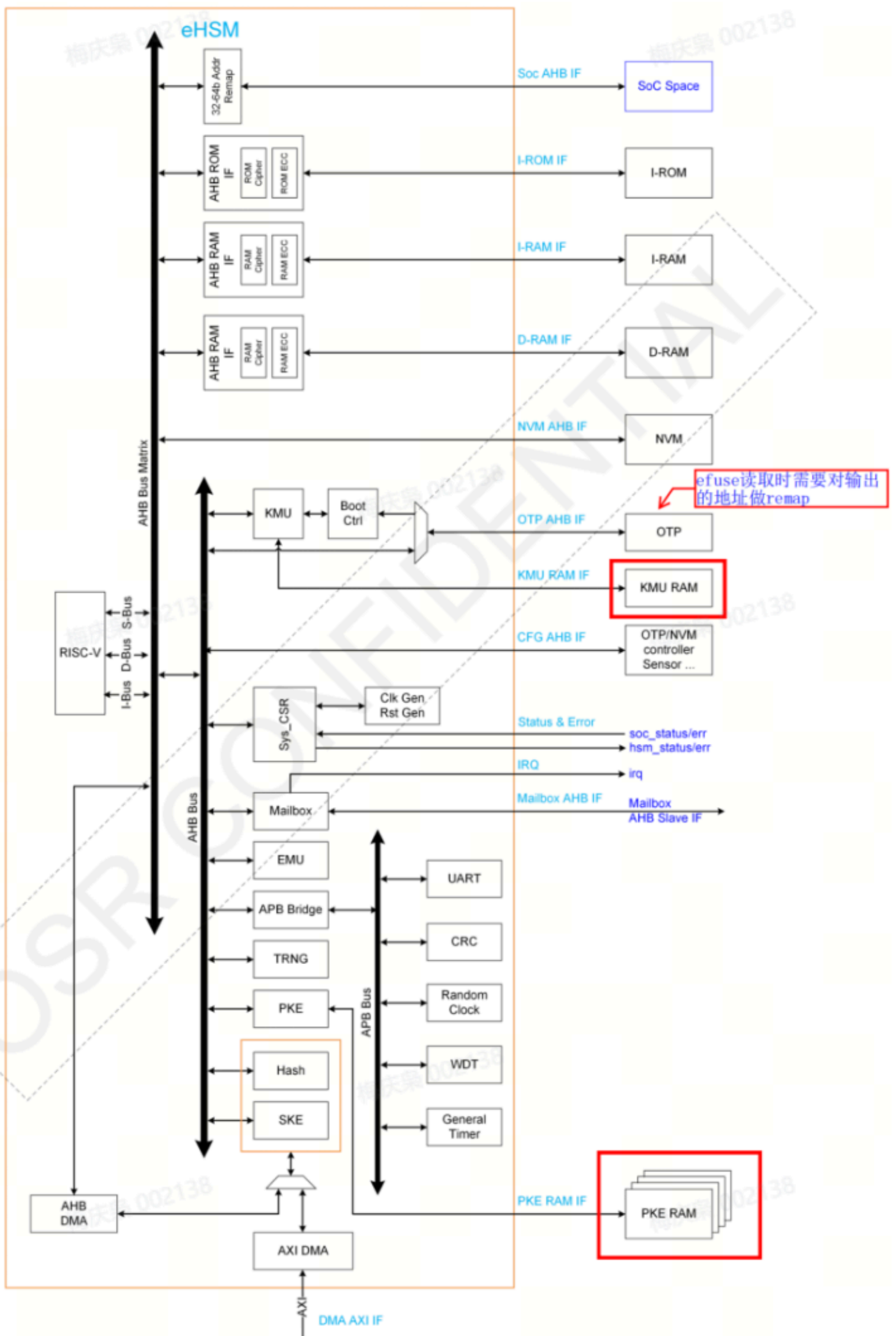
说明：访问soc\_mem的AHB32\_Master/AXI\_Master接口地址位都是64位，另外AHB是single访问，AXI支持预约制的outstanding访问；

eHSM功能包括：

- 设备信任根
- 安全启动
- 安全升级
- 安全存储
- 密钥管理
- 固件验签
- 生命周期管理
- 安全调试
- 密码服务
- 错误管理
- 外设

等。

eHSM的内部结构可参考下图： -





- 以上为eHSM的结构。
- eHSM：包含一组jtag接口，用于调试eHSM的riscv核
- 包含sram接口，用于对接ehsm所要用的5组sram
- 5组SRAM：64KB IROM（存放bootloader）、64K SRAM（存放CPU数据）、256KB IRAM(运行固件)、1KB KMU\_RAM(密钥管理)、12KB SRAM（公钥运算空间，与实现算法相关）
- 包括对外通信的soc\_ahb\_if/DMA\_AXI\_IF接口，用于访问soc memory空间
- 包含AHB slave IF，用于mailbox对外通信
- CFG AHB IF为冗余接口。
- 包含一组OTP AHB IF，用于读取efuse信息。

## 7.3 安全efuse

LCS：生命周期状态

**efuse烧写：**

1. eHSM需要一个efuse，此efuse只能eHSM访问。
2. efuse可通过jtag 烧写，可通过调用eHSM烧写。
3. LCS=USER阶段，默认不可使用jtag烧写。
4. LCS=TEST/DEV/MANU阶段，可使用JTAG或者eHSM烧写。
5. 出厂测试白片阶段本身无安全约束，允许开放OTP写权限，在此过程中调eHSM是为避免密钥泄露，不强制要求只有通过eHSM进行烧写，该阶段OTP有自己的测试接口，通过测试接口烧写也可以
6. 有两个烧写通道，一个通过JTAG口，一个通过eHSM，ATE烧写时可使用JTAG接口也可使用eHSM，此阶段不强制要求eHSM（推荐使用eHSM），不使用eHSM安全性由外部设备保证

**efuse访问限制：**

1. eHSM所需efuse信息存储于安全efuse中，只有eHSM可访问。
2. Efuse controller提供autoload功能，可autoload出LCS、UID信息供管理子系统使用。
3. otp接口需要在auto load done之前，反压ahb接口，等到安全 efuse ready再释放ehsm复位

**ehsm所需的efuse信息为：**

传感器错误响应，可以不用

Table 34: OTP 内存映射地址

| Field                           | Offset                    | Size/Word | Multi-update | Decoded by |
|---------------------------------|---------------------------|-----------|--------------|------------|
| Life Cycle 4                    | 0x000                     | 1         | yes          | HW         |
| UID                             | 0x004                     | 5         | no           | FW         |
| HW Control Table 35             | 0x018                     | 2         | yes          | HW         |
| FW Control-eHSM                 | 0x020                     | 2         | yes          | HW         |
| FW Control-SoC                  | 0x028                     | 2         | yes          | HW         |
| Error Response Control Table 36 | 0x030                     | 8         | no           | FW         |
| eHSM Version Counter            | 0x050                     | 4         | yes          | FW         |
| SOC Version Counter             | 0x060                     | 4         | yes          | FW         |
| OTP Key N Attribute 3.4.3       | 0x070+<br>0x4*0xA*N       | 1         | yes          | HW         |
| OTP Key N 3.4.1                 | 0x070+<br>0x4*(0xA*N+0x1) | 8         | no           | HW         |
| OTP Key N CRC Table 38          | 0x070+<br>0x4*(0xA*N+0x9) | 1         | no           | HW         |

Table 42: OTP 密钥安装

| OTP Key Name             | Type                     | Owner         | Burned by     | Key from  | Time to Burn | Install Flow                                                                  |
|--------------------------|--------------------------|---------------|---------------|-----------|--------------|-------------------------------------------------------------------------------|
| Chip Root Key            | Symmetric                | Chip Vendor   | Chip Vendor   | eHSM TRNG | TEST/DEV     | 加密由 TRNG 生成的随机数, 并烧录到 OTP 中。                                                  |
| Device Root Key          | Symmetric                | Device Vendor | Device Vendor | eHSM TRNG | MANU         | 加密由 TRNG 生成的随机数, 并烧录到 OTP 中。                                                  |
| Secret Key 0             | Symmetric                | Chip Vendor   | Chip Vendor   | eHSM TRNG | TEST/DEV     | 加密由 TRNG 生成的随机数, 并烧录到 OTP 中。                                                  |
| eHSM Debug Key           | Asymmetric               | Chip Vendor   | Chip Vendor   | External  | TEST/DEV     | 导入公钥哈希值, 由 Chip Root Key 加密, 烧录到 OTP 中。                                       |
| eHSM FW Verify Key       | Asymmetric/<br>Symmetric | Chip Vendor   | Chip Vendor   | External  | TEST/DEV     | 非对称: 导入公钥哈希值, 由 Chip Root Key 加密, 烧录到 OTP 中。<br>对称: 直接将 TRNG 的随机数据烧录到 OTP 中   |
| eHSM Encrypt Key         | Symmetric                | Chip Vendor   | Chip Vendor   | eHSM TRNG | TEST/DEV     | 加密由 TRNG 生成的随机数, 并烧录到 OTP 中。                                                  |
| eHSM Upgrade Encrypt Key | Symmetric                | Chip Vendor   | Chip Vendor   | External  | TEST/DEV     | 导入密钥值, 由 Chip Root Key 加密, 烧录到 OTP 中。                                         |
| eHSM Upgrade Verify Key  | Asymmetric/<br>Symmetric | Chip Vendor   | Chip Vendor   | External  | TEST/DEV     | 非对称: 导入公钥哈希值, 由 Chip Root Key 加密, 烧录到 OTP 中。<br>对称: 直接将 TRNG 的随机数据烧录到 OTP 中   |
| eHSM Private Key         | Asymmetric               | Chip Vendor   | Chip Vendor   | eHSM TRNG | TEST/DEV     | 加密由 PKE 算法生成的私钥, 并烧录到 OTP 中。                                                  |
| SoC Debug Key            | Asymmetric               | Device Vendor | Device Vendor | External  | MANU         | 导入公钥哈希值, 由 Device Root Key 加密, 烧录到 OTP 中。                                     |
| SoC FW Verify Key        | Asymmetric/<br>Symmetric | Device Vendor | Device Vendor | External  | MANU         | 非对称: 导入公钥哈希值, 由 Device Root Key 加密, 烧录到 OTP 中。<br>对称: 直接将 TRNG 的随机数据烧录到 OTP 中 |



Table 42: OTP 密钥安装

| OTP Key Name            | Type                     | Owner         | Burned by     | Key from  | Time to Burn | Install Flow                                                                      |
|-------------------------|--------------------------|---------------|---------------|-----------|--------------|-----------------------------------------------------------------------------------|
| SoC Encrypt Key         | Symmetric                | Device Vendor | Device Vendor | eHSM TRNG | MANU         | 加密由 TRNG 生成的随机数, 并烧录到 OTP 中。                                                      |
| SoC Upgrade Encrypt Key | Symmetric                | Device Vendor | Device Vendor | External  | MANU         | 导入 <b>密钥</b> 值, 由 Device Root Key, 烧录到 OTP 中。                                     |
| SoC Upgrade Verify Key  | Asymmetric/<br>Symmetric | Device Vendor | Device Vendor | External  | MANU         | 非对称: 导入公钥哈希值,<br>由 Device Root Key 加密, 烧录到 OTP 中。<br>对称: 直接将 TRNG 的随机数据烧录到 OTP 中。 |
| SoC Private Key         | Asymmetric               | Device Vendor | Device Vendor | eHSM TRNG | MANU         | 加密由 PKE 算法生成的私钥, 并烧录到 OTP 中。                                                      |
| User Auth Key           | Asymmetric               | User          | Device Vendor | External  | MANU         | 导入公钥哈希值, 由 Device Root Key 加密, 烧录到 OTP 中。                                         |

efuse的排布信息参考:

NGU800P Efuse信息汇总

| 大类     | 小类   | 烧写项目               | Redundancy信息预留                                      | 烧写阶段   | 使用方式                 | 编码规则  | Bit数目 | 安全性 | Valid标识 | Lock标识 |
|--------|------|--------------------|-----------------------------------------------------|--------|----------------------|-------|-------|-----|---------|--------|
| 芯片身份标识 | ECID | Lot ID             |                                                     | CP     | Shadow Register Read | ASCII | 56    | N   | Y       | Y      |
|        |      | Wafer ID           | Dynamic, 0~25 数字                                    | CP     | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        |      | X_coord            | Dynamic, 0~30                                       | CP     | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        |      | Y_coord            | Dynamic, 0~30                                       | CP     | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        |      | Asse House         | 封装厂                                                 | FT     | Shadow Register Read | ASCII | 8     | N   | Y       | Y      |
|        |      | Assembly Date Code | Dynamic                                             | FT     | Shadow Register Read | ASCII | 64    | N   | Y       | Y      |
|        |      | Fab ID             | 0x0 = HL                                            | CP/SLT | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        |      | Sample 类型          | 0x01=Engineering Sample,<br>0x02=Production Sample, | FT     | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        |      | Silicon Rev        | 0x0= Rev1.0                                         | CP/SLT | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        |      | Burn-IN            | 0x0 = Burn In                                       | FT     | Shadow Register Read | DEC   | 8     | N   | Y       | Y      |
|        | SALT | 芯片SALT值            |                                                     |        |                      | 64    | N     | Y   | Y       |        |

| 1  | 大类   | 小类        | 烧写项目                                | Redundancy信息预留                                                                                                                                    | 烧写阶段      | 使用方式                 | 编码规则        | Bit数目 | 安全性 | Valid标识 | Lock标识 |
|----|------|-----------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------------------|-------------|-------|-----|---------|--------|
| 43 | 安全信息 | 安全启动与生命周期 | Lifecycle State                     | 生命周期<br>TEST: 0x000000<br>DEV: 0x42818414<br>MANU: 0x46C1A416<br>USER: 0x57C1EC96<br>DEBUG: 0xD7C5FCDE<br>DESTROY: 0xFFFFFFFF<br>UNDEFINE: others | FT/ONLINE | Hardware Output      | Thermometer | 32    | Y   | Y       | N      |
| 44 |      |           | UID                                 | 芯片Unique ID                                                                                                                                       | CP        | Hardware Output      | BIN         | 160   | Y   | Y       | Y      |
| 45 |      |           | hw_control                          | OTP 硬件控制字段<br>字段和软件确认后给一个烧写内容                                                                                                                     | FT        | Shadow Register Read |             | 64    | Y   | Y       | Y      |
| 46 |      |           | fw control-eHSM                     | 需要参考《OSR_eHSM_BootLoader_TRM_vx.x.pdf》@梅庆东                                                                                                        | FT        | Shadow Register Read |             | 64    | Y   |         |        |
| 47 |      |           | fw control-soc                      | 需要参考《OSR_eHSM_BootLoader_TRM_vx.x.pdf》@梅庆东                                                                                                        | FT        | Shadow Register Read |             | 64    | Y   |         |        |
| 48 |      |           | eHSM Version Counter                | eHSM 内部Firmware的版本Counter                                                                                                                         | FT/ONLINE | Shadow Register Read |             | 128   | Y   |         | N      |
| 49 |      |           | SOC Version Counter                 | C908 Firmware的版本Counter                                                                                                                           | FT/ONLINE | Shadow Register Read |             | 128   | Y   |         | N      |
| 50 |      |           | patch_cfg                           | eHSM内部CPU ROM的Patch, 大小特定                                                                                                                         | FT        | Shadow Register Read |             | 32    | Y   | Y       | Y      |
| 51 |      |           | patch_addr                          | eHSM内部CPU ROM的Patch, 大小特定                                                                                                                         | FT        | Shadow Register Read |             | 256   | Y   |         |        |
| 52 |      |           | patch_data                          | eHSM内部CPU ROM的Patch, 大小特定                                                                                                                         | FT        | Shadow Register Read |             | 512   | Y   |         |        |
| 53 |      |           | OTP Key Attribute+OTP key+OTP K CRC | 密钥属性+密钥+密钥CRC值<br>一套占用320, 共计需要14套                                                                                                                | FT        | Shadow Register Read |             | 4480  | Y   |         |        |

UID 需求专题讨论

UID定义

UID是芯片唯一标识，需要每个芯片不一样，UID存储于efuse，共160bit。其中前 128 bit为 UID 数据，后 32bit 为前 128 bit 的 CRC32 校验。生产时应确保每个 eHSM 写入的 128 bit UID 是唯一的，否则一些鉴权操作会存在安全风险。

其中 CRC32 算法的多项式为：0x04C11DB7，初始值为 0xFFFFFFFF。

UID 由软件读取并使用。

UID由多部分组合生成（TBD 待讨论）：

代码块

```
1  UID input = ECID[160] || TRNG [256]
2  UID = Truncate128[hash[ECID||TRNG]]+CRC
```

UID烧写

代码块

```
1  KMS/SW TRNG
2  ↓
3  128-bit Random+[ECID]+CRC
4  ↓
5  Burn to eFuse
```

UID为什么添加随机数：

使用随机数通过把碰撞概率降到极低，达到工程上可接受的“唯一”。

使用真随机数作为UID，需满足：

TRNG经过标准验证，如：

- NIST SP 800-90B / SP 800-90C

- AIS31

## 概率论支持

- TRNG产生的 UID，量产碰撞概率可忽略。

# 8. 安全子系统功能

## 8.1 安全启动

此功能提供镜像校验功能，在整个系统运行中，soc侧通过mailbox机制将镜像信息传递给eHSM，eHSM对镜像进行校验。通过校验镜像的完整性和合法性，可确保只有经过授权且未被篡改的镜像能够在系统中运行。从而有效防止恶意软件的注入和非法篡改，保证系统的安全性和稳定性。

### 启动链建议

C908 BootROM → eHSM FW → 其他 FW

要求：

- 启动链全链路签名校验
- 支持国密/国际算法，校验算法支持：
  - a. 支持国密算法：SM2-with-SM3-SM4（推荐）
  - b. 支持国际算法：ECDSA-P384-with-SHA384-AES
  - c. 支持国际算法：ECDSA-P256-with-SHA256-AES（推荐）
  - d. 支持国际算法：ECDSA-P521-with-SHA512-AES
  - e. 支持RSA算法：RSA-4096-with-SHA512-AES（推荐）
  - f. 支持RSA算法：RSA-2048-with-SHA256-AES
- 校验失败停止启动

### Secure Boot主要提供功能：

- 解析镜像格式，根据公钥HASH算法,镜像签名算法完成镜像验签
- 解析镜像格式，根据镜像加密算法、加密Flag，选择是否镜像解密
- 解析镜像格式，根据镜像版本和OTP中version counter对比，判断是否可以执行

### eHSM的启动流程：

eHSM 安全启动是 SOC 芯片启动过程的一部分，是 SOC 芯片建立安全信任根的必要步骤，启动过程如下图所示。



图 SOC 芯片安全启动过程

### 1. 芯片上电初始化

芯片上电后首先对芯片进行配置和初始化，使得芯片达到可以稳定运行的条件。芯片初始化完成后释放 eHSM Reset，进入 eHSM 硬件启动阶段。

### 2. eHSM 硬件 Boot

eHSM 硬件 Boot 主要是完成 security 相关的自检和配置，包括随机数模块自检、存储器加解密等模块的随机密钥初始化、自 OTP 中读取生命周期、密钥解密等。

### 3. eHSM Bootloader 执行

eHSM Bootloader 主要完成密码算法自检、eHSM FW 校验。

### 4. eHSM FW 执行

eHSM 固件（FW）主要完成 SOC Bootloader 的校验（可选），之后等待接收 Mailbox 指令。

### 5. SOC Bootloader 执行

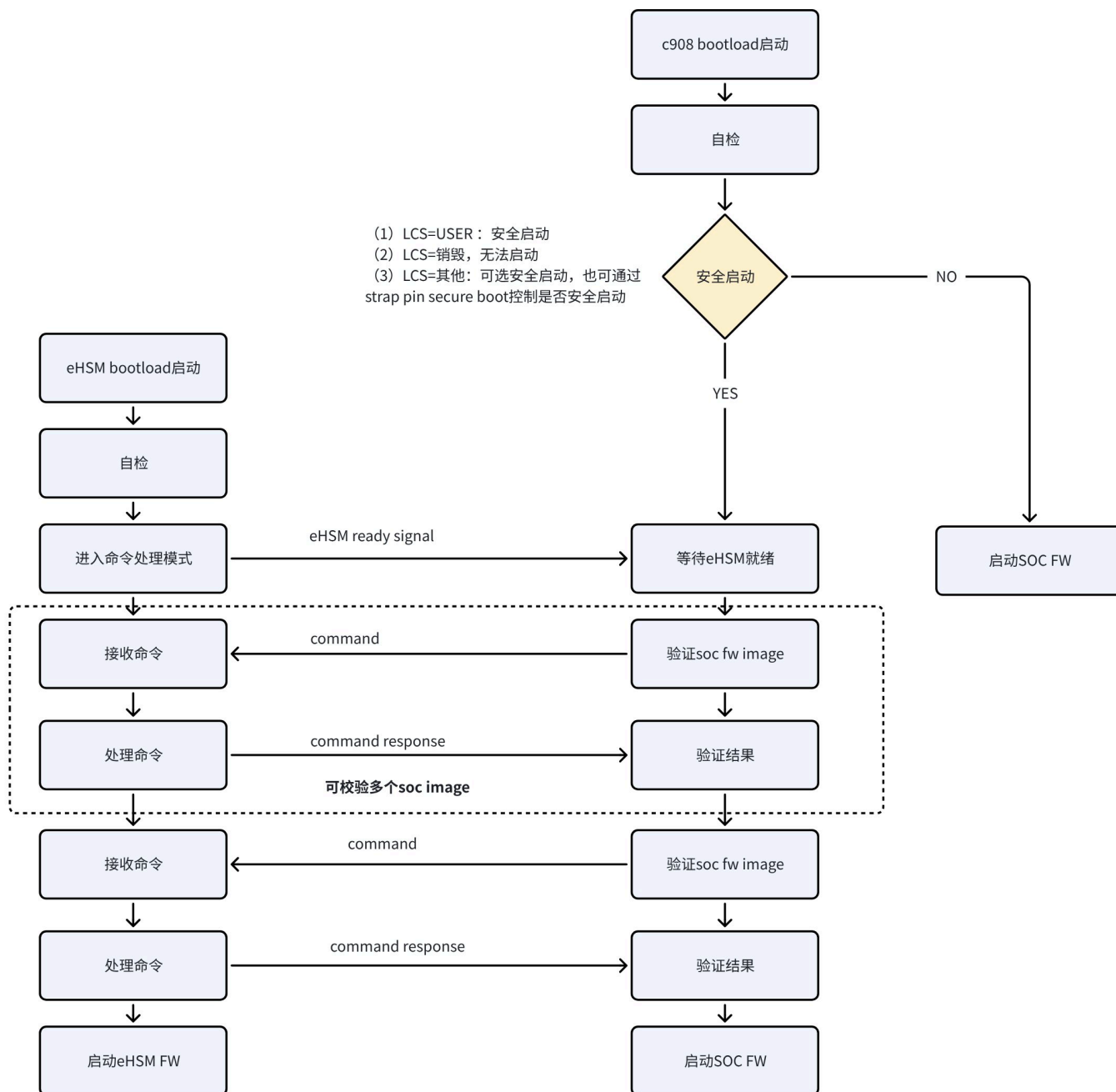
SOC Bootloader 完成与 SOC 相关的初始化配置，并调用 eHSM FW 对 SOC FW 进行校验（可选）。

### 6. SOC FW 执行

- 执行与 SOC 应用相关的操作。
- eHSM 对 SOC Bootloader 和固件的校验算法支持对称算法和公钥算法，依据所支持的算法和校验对象的 code size 大小，启动时间有所不同。
- 为加快 SOC 芯片启动过程，可采用 eHSM 与 SOC Bootloader 并行启动的方式

**SOC安全启动流程如下：**



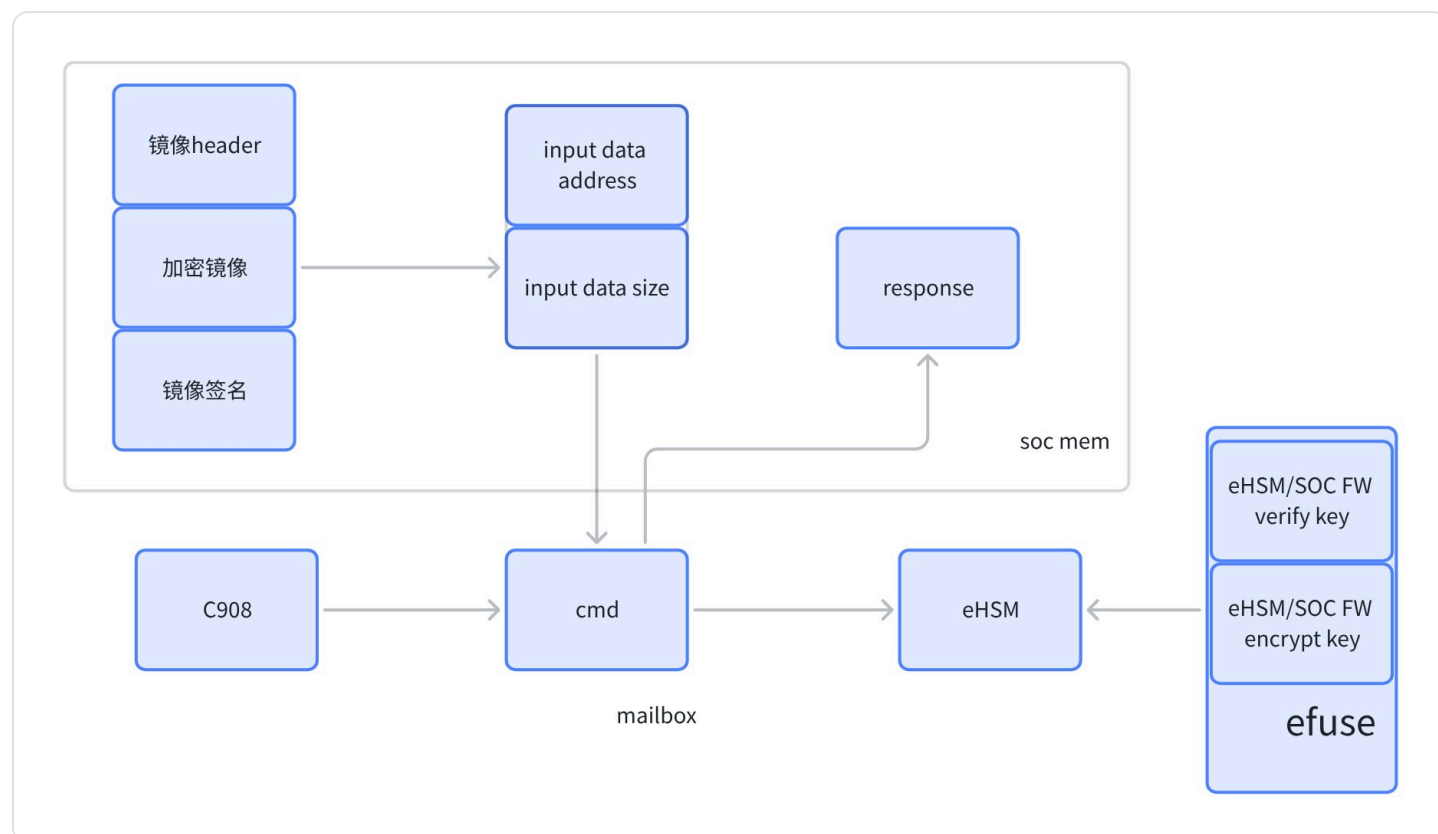


## 启动顺序说明：

1. 上电后管理子系统功耗C908最先复位释放，然后运行SPI或者UART Boot导入的镜像文件
2. 解复位eHSM，等待eHSM硬件 boot成功
3. 等待efuse autolod 成功，并完成LCS状态机更新（LCS进入有效状态后再作为后续判断依据）
4. 管理子系统C908 上电之后，固件使用LCS（生命周期状态）作为判断条件，判断是否执行安全启动。
  - LCS=USER 强制触发安全启动
  - LCS=其他阶段，可选触发安全启动。（1）可由软件可选触发安全启动（2）可由strap pin secure boot 控制是否启动安全启动。

5. 如需安全启动，管理子系统中的C908等待eHSM硬件boot完成，收到eHSM ready信号后，C908启动核负责搬运eHSM镜像到eHSM sram，eHSM启动完成再逐次获取写入到各级微核的镜像文件，做完验签和解密后，上传到对应的RAM空间，并释放对应的MCU复位
6. 如果验签或者解密不通过，管理子系统中的C908会做好记录，并不解复位对应的MCU

### eHSM解密验签流程：



1. 管理子系统C908通过mailbox将镜像搬运到eHSM
2. 使用对称算法解密镜像，得到明文镜像
3. eHSM解析镜像，获取公钥，对公钥进行hash，和otp里存储的eHSM FW verify key比对，比对通过则公钥合法，并进行counter校验：
4. 读取镜像携带的版本counter，与OTP/安全存储中的当前counter值进行比对，必须满足镜像  $\text{counter} \geq \text{安全存储counter} + 1$ ，否则判定为回滚或重放攻击，直接失败
5. 使用比对通过的公钥验签镜像
6. 完成镜像的验签解密。

## 8.2 安全升级

满足客户关于升级可信根、禁止绕过升级路径、防止不可恢复和 bootkit 的要求。

### 升级原则

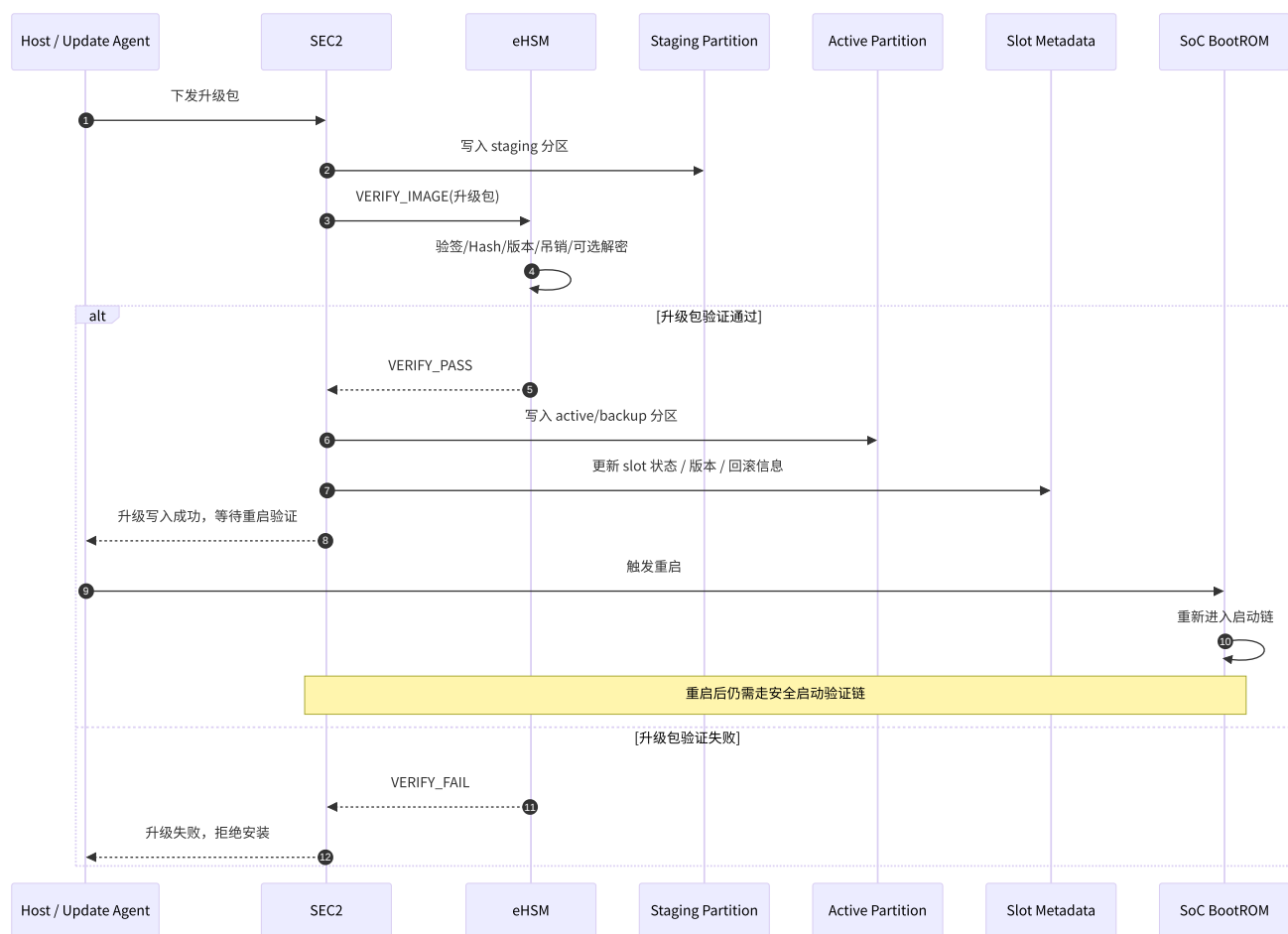


1. 所有升级镜像必须由C908调用 eHSM 校验
2. Host 只能将升级包写入缓冲区
3. 启动关键区不得允许 Host 直接写入
4. 需要支持 recovery / backup / rollback
5. 需要支持版本控制与 anti-rollback

## 升级流程

1. Host 通过 PLDM 或自定义通道将升级包发送给C908 (安全域外 SRAM 或 DDR region)
2. C908将镜像写入缓冲区
3. C908调用 eHSM:
  - 验签
  - HASH 校验
  - 根据镜像的加解密字段必要时解密
4. 校验通过后, C908才允许写入 active / backup 分区
5. 更新 slot metadata / counter / version policy , 更新前必须执行counter比对: 读取镜像携带的 counter (版本递增计数) , 与OTP中的当前anti-rollback counter进行比较, 必须满足镜像  $\text{counter} \geq \text{当前otp存储counter} + 1$  , 否则拒绝更新; 更新成功后将otp存储counter同步更新为镜像counter, 防止旧镜像重放与回滚攻击
6. 重启后重新进入安全启动链验证
7. 若新镜像失败, 则回滚至旧镜像

Host API 中的 `ehsm_verify_image` 和 `ehsm_upgrade_fw_image_*` 可作为这类能力的底层支持。



## 8.3 生命周期（LCS）管理

芯片全生命周期划分为 6 个阶段，生命周期标识占 4Byte 空间，存储在 efuse 中。生命周期每个阶段定义的权限不同，同时兼顾易用性和安全性。生命周期按芯片生产、开发、使用等实际过程逐级向后跳转，但不支持生命周期回退。

具体定义见下表。

| 生命周期             | 简称   | 管理员           | 描述                              |
|------------------|------|---------------|---------------------------------|
| TEST_MODE        | TEST | Chip Vendor   | 测试模式，是芯片的生产及 CP/FT 等筛选测试阶段      |
| DEVELOP_MODE     | DEVE | Chip Vendor   | 开发模式，是芯片固件开发和样片测试阶段             |
| MANUFACTURE_MODE | MANU | DEVICE Vendor | 设备开发模式，是芯片客户如设备厂商基于芯片做设备开发和调试阶段 |
| USER_MODE        | USER | Customer      | 用户模式，是终端用户使用阶段                  |
| DEBUG_MODE       | DEBG | Chip Vendor   | 调试模式，是芯片返厂做失效和问题分析阶段            |
| DESTROY_MODE     | DEST | -             | 自毁模式，是芯片进入废弃、销毁的阶段              |
| UNDEFINE_MODE    | -    | -             | 未定义模式，等同于自毁模式                   |

生命周期值：

Table 45: 生命周期 (不带 ECC) 定义 (OTP 位的默认值为 0 的情形)

| Life Cycle       | Life Cycle Value |
|------------------|------------------|
| TEST_MODE        | 0x00000000       |
| DEVELOP_MODE     | 0x42818414       |
| MANUFACTURE_MODE | 0x46C1A416       |
| USER_MODE        | 0x57C1EC96       |
| DEBUG_MODE       | 0xD7C5FCDE       |
| DESTROY_MODE     | 0xFFFFFFFF       |
| UNDEFINE_MODE    | Others           |

生命周期值：

- 存储于安全efuse，共32bit
- eHSM可访问efuse中的生命周期值
- 管理子系统可访问生命周期 autoload值

LCS Lifecycle Control：

- 盲片→TEST → DEV → MANU → USER → DEBUG → DESTROY

LCS Lifecycle 烧写：

- 支持JTAG烧写，支持在线烧写

- USER → DEBUG：使用User Auth key 鉴权之后，允许烧写
- 其他LCS状态跳转不需要鉴权

盲片阶段：

- 检查基本功能是否正常
- 默认可上电启动，BootROM可工作，可做基本boot验证，可做测试
- debug接口可正常使用
- 增加区分的界限，安全efuse LCS valid 信号=0，则为盲片

TEST\_MODE：

- 测试模式，为LCS的第一个阶段，已经进入芯片生命周期管理
- TEST\_MODE应在otp autoload之后生效。
- eHSM的LCS工作在eHSM的复位i\_rst域，eHSM的LCS应在otp autoload生效且i\_rst解复位之后生效。
- TEST\_MODE可选不启动eHSM
- 随机数测试应在TEST\_MODE下进行，在TEST\_MODE可以不启动eHSM，但随机数测试可以启动。

DEV\_MODE：DEV\_MODE下可选不启动eHSM。

USER\_MODE：USER可能暴露在非安全环境中，强制安全启动

DEBG\_MODE：返厂维修调试模式，符合预期的调试接口可在此阶段经过鉴权打开。

DEST\_MODE: 销毁模式，不提供安全服务，eHSM无法启动，需要销毁密钥。

UNDEF：eHSM 启动失败，不提供安全服务

芯片内受生命周期管控的服务如下：

| function       | TEST | DEV    | MANU   | USER   | DEBG   | DEST | 描述                        |
|----------------|------|--------|--------|--------|--------|------|---------------------------|
| Secure Service | ON   | ON     | ON     | ON     | ON     | OFF  | eHSM提供的安全服务，包括加解密，密钥管理等服务 |
| Soc debug en   | ON   | SW-OFF | SW-OFF | SW-OFF | SW-OFF | OFF  |                           |
| secure_jtag_en | OFF  | ON     | ON     | SW-ON  | SW-ON  | OFF  |                           |
|                | OFF  | ON     | ON     | SW-ON  | SW-ON  | OFF  |                           |

|               |    |        |        |        |        |     |          |
|---------------|----|--------|--------|--------|--------|-----|----------|
| firewall_en   |    |        |        |        |        |     |          |
| eHSM Debug en | ON | SW-OFF | SW-OFF | SW-OFF | SW-OFF | OFF | eHSM内部使用 |
| PCle_debug_en | ON | ON     | ON     | SW-ON  | SW-ON  | OFF |          |
| uart_en       | ON | ON     | ON     | OFF    | OFF    | OFF |          |

- ON : 默认启用，无法通过软件禁用
- OFF : 默认禁用，无法通过软件启用
- SW-ON : 默认禁用，可以通过软件启用
- SW-OFF : 默认启用，可以通过软件禁用
- OTP-ON : 默认禁用，可以通过 OTP 可选控制位启用
- OTP-OFF : 默认启用，可以通过 OTP 可选控制位禁用

### 8.3.1 调试及接口安全

eHSM支持 Mailbox 鉴权通道。SOC可通过管理子系统内的C908访问eHSM Mailbox的接口作为鉴权通道；

安全调试鉴权支持非对称算法。

功能：

- JTAG Gating
- Firewall gating
- Pcie debug gating
- Debug Authentication

eHSM提供的LCS管控服务如下，仅供参考：

Table 46: 生命周期中的功能控制

| Function          | TEST | DEVE    | MANU    | USER    | DEBG    | DEST |
|-------------------|------|---------|---------|---------|---------|------|
| Secure Service    | ON   | ON      | ON      | ON      | ON      | OFF  |
| SoC Secure Debug  | ON   | SW-OFF  | SW-OFF  | SW-ON   | SW-ON   | OFF  |
| eHSM Secure Debug | ON   | SW-OFF  | SW-ON   | SW-ON   | SW-ON   | OFF  |
| TRNG Health-test  | ON   | OTP-OFF | OTP-OFF | OTP-OFF | OTP-OFF | OFF  |

其中：eHSM Secure Debug/TRNG Health-test为eHSM内部使用。

鉴权流程（非对称算法）：

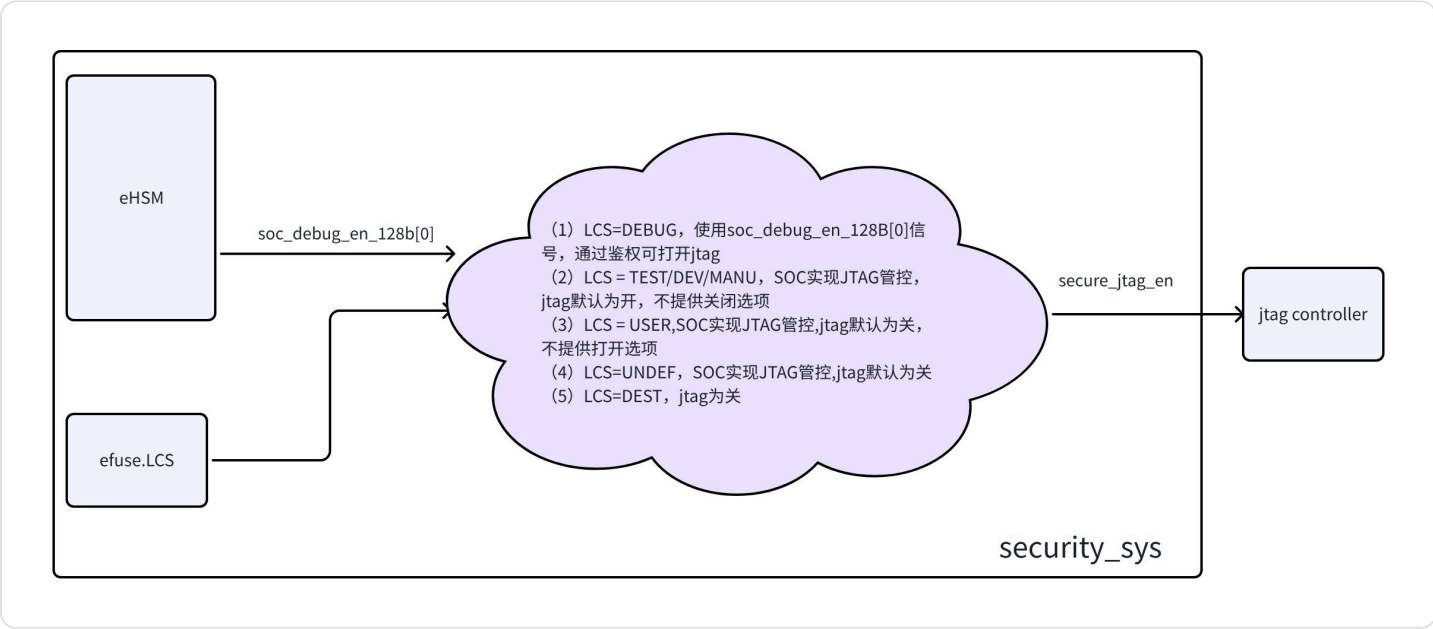
- 1. 通过PCIE给C908发送鉴权指令。
- 2. C908发送请求给 eHSM ，通过eHSM mailbox发送调试鉴权cmd给eHSM
- 3. eHSM 输出挑战值{随机数||UID} ，放到soc memory
- 4. 鉴权设备对挑战值计算签名，并将签名发送回eHSM
  - 非对称算法：使用 ehsm 调试密钥/soc 调试密钥的相应私钥签名。
- 5. eHSM 校验签名，使用 ehsm 调试密钥/soc 调试密钥验证签名，一致则开启调试接口，并送出 o\_ehsm\_status状态指示

8.3.1.1 安全JTAG

JTAG受LCS管控的行为如下：

| LCS     | UNDEF | TEST | DEV | MANU | USER | DEBUG | DEST |
|---------|-------|------|-----|------|------|-------|------|
| JTAG 值  | OFF   | ON   | ON  | ON   | OFF  | SW_ON | OFF  |
| JTAG默认值 | OFF   | ON   | ON  | ON   | OFF  | OFF   | OFF  |

JTAG受LCS管控的逻辑如下：



- (1) LCS = debug，使用soc\_debug\_en\_128b[0]信号控制jtag通路，默认jtag关闭，鉴权设备发送鉴权请求给eHSM，鉴权通过则打开jtag，失败则保持关闭
- (2) LCS = TEST、DEV、MANU，使用SOC实现jtag控制通路，默认jtag打开，不提供关闭选项
- (3) LCS=USER， SOC实现JTAG管控，默认jtag关闭，不提供打开选项
- (4) LCS=UNDEF， jtag关闭

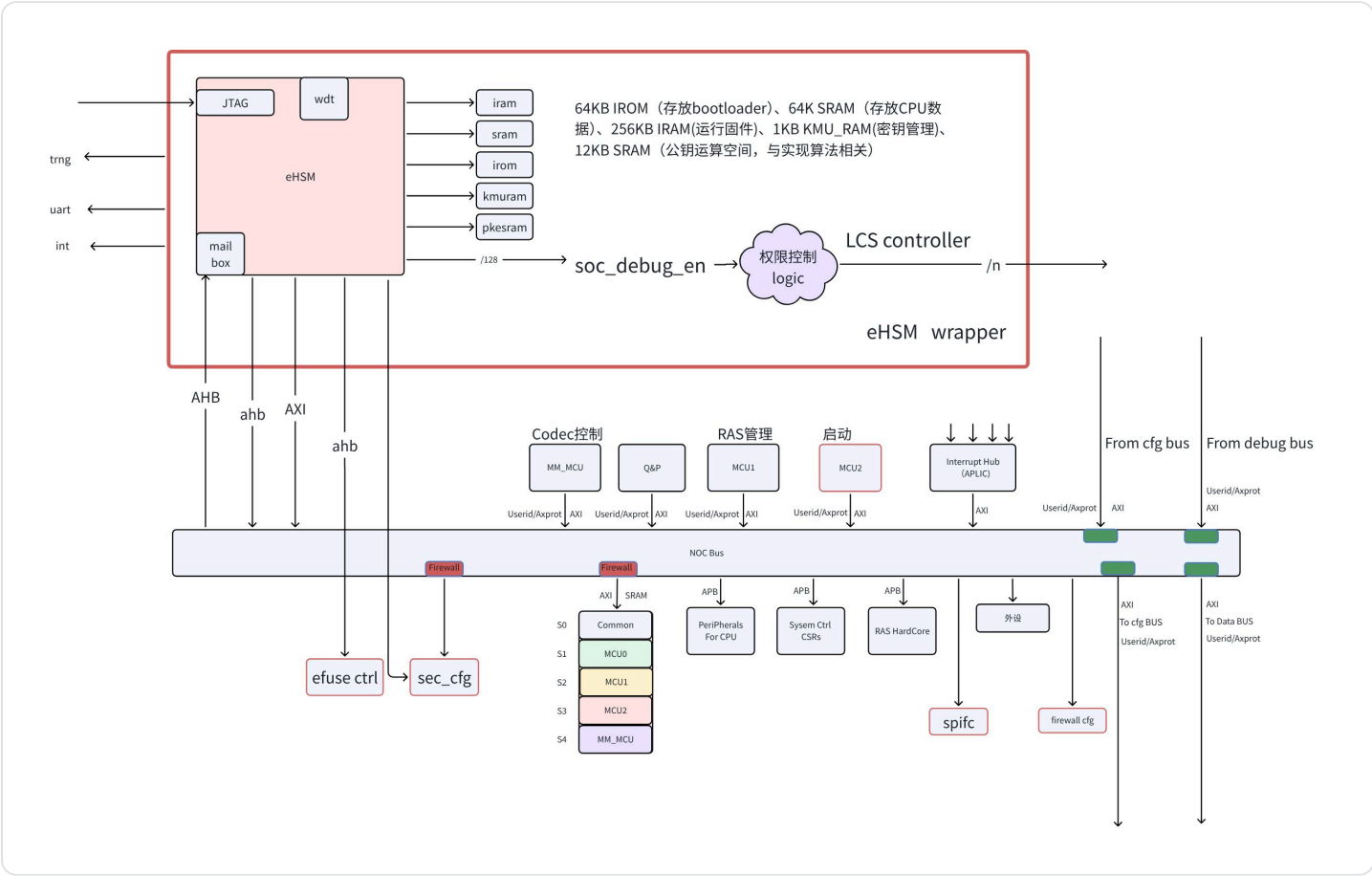
(5) LCS=DEST, jtag 关闭

eHSM wrapper送出1bit secure\_jtag\_en信号到jtag\_controller，用于钳位jtag的TCK/TDI/TMS 信号

8.3.1.2 Firewall 安全管控

方案背景：

全芯片的Firewall 只能通过管理子系统的C908控制改配。C908的firmware经过验签，可认为是受信任的。因此firewall管控的目标为：防范合法但无意或者恶意的配置修改风险，仅有验签过的固件控制，安全控制集中在软件侧。验签保证“代码来源可信”，但 firewall 配置不做安全管控时，安全性完全依赖软件正确性与密钥体系，硬件侧无安全保障。



Firewall\_en 控制firewall的配置权限。ON=firewall配置可以自由改配，OFF=firewall不能修改配置。  
SW\_ON=经过鉴权后，firewall配置可修改。

Firewall\_en LCS管控的行为如下：

| LCS             | UNDEF | TEST | DEV | MANU | USER  | DEBUG | DEST |
|-----------------|-------|------|-----|------|-------|-------|------|
| firewall_en 值   | OFF   | ON   | ON  | ON   | SW_ON | SW_ON | OFF  |
| firewall_en 默认值 | OFF   | ON   | ON  | ON   | OFF   | OFF   | OFF  |

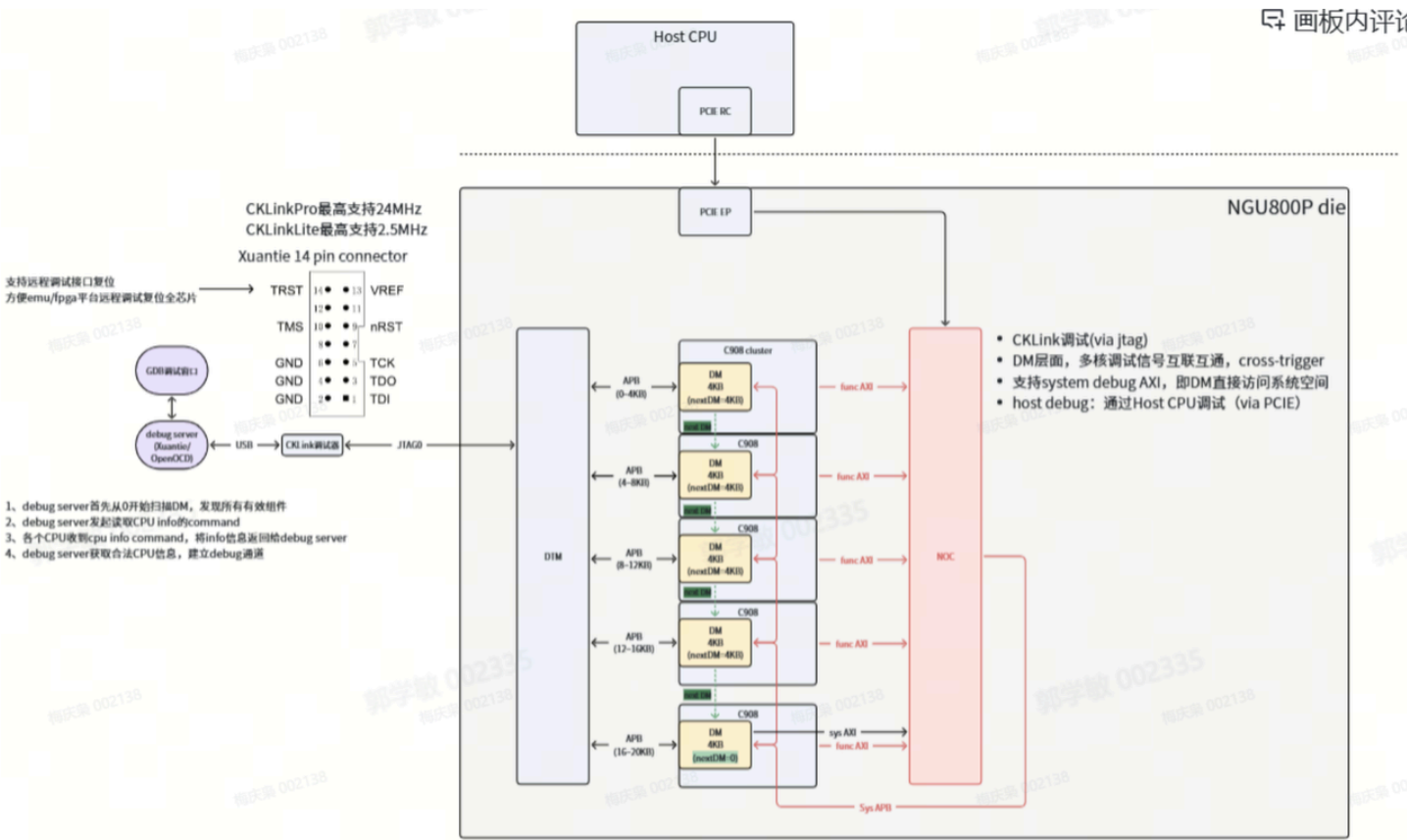
SW-ON：软件可配，SW通过鉴权可打开firewall 配置权限。

8.3.1.3 PCIe 访问限制

敏感信息，软件限制无通路访问。敏感信息：efuse/固件运行rtos sram/运行linux dram/pcie remap (IATU/apt remap)

8.3.1.4 PCIe debug安全管控

Host CPU可通过pcie 对riscv cpu进行调试。  
可通过PCIe访问debug寄存器，eHSM根据LCS限制对core的debug权限  
非debug通路，不做限制。



PCIe debug LCS管控的行为如下：

| LCS                   | UNDEF | TEST | DEV | MANU | USER  | DEBUG | DEST |
|-----------------------|-------|------|-----|------|-------|-------|------|
| PCIe_debug_e<br>n 值   | OFF   | ON   | ON  | ON   | SW_ON | SW_ON | OFF  |
| PCIe_debug_e<br>n 默认值 | OFF   | ON   | ON  | ON   | OFF   | OFF   | OFF  |



- (1) LCS = debug，使用soc\_debug\_en\_128b[3]信号控制pcie调试通路，默认pcie调试通路关闭，鉴权设备发送鉴权请求给eHSM，鉴权通过则打开pcie debug，失败则保持关闭
- (2) LCS = TEST、DEV、MANU，使用SOC实现jtag控制通路，默认pcie debug打开，不提供关闭选项
- (3) LCS=USER，默认jtag关闭，软件发起鉴权请求，鉴权通过则打开pcie debug，失败则保持关闭
- (4) LCS=UNDEF，pcie debug 通路关闭
- (5) LCS=DEST，pcie debug 通路关闭

eHSM wrapper送出1bit pcie\_debug\_en信号到PCIE，用于钳位PCIE的调试通路

8.3.1.5 UART 安全控制

PCIE标卡通过B-to-L连接器对外提供UART\_DIE0，UART\_DIE1调试通道。

Uart LCS=user，做gating，不开uart

uart LCS管控的行为如下：

| LCS         | UNDEF | TEST | DEV | MANU | USER  | DEBUG | DEST |
|-------------|-------|------|-----|------|-------|-------|------|
| uart_en 值   | OFF   | ON   | ON  | ON   | SW_ON | SW_ON | OFF  |
| uart_en 默认值 | OFF   | ON   | ON  | ON   | OFF   | OFF   | OFF  |

SW-ON：软件鉴权可打开uart

8.3.1.6 DFT/SCAN 安全控制（DFT需求待定）

DFT 通过test\_mode PAD口切换到DFT 测试模式，DFT链可访问芯片的sram，efuse等关键信息，因此DFT根据LCS做安全管控。

DFT scan chain的使用阶段：

- 1. USER 阶段不会使用
- 2. DEBUG 阶段 scan dump如没有需求则不使用
- 3. 其他LCS阶段默认打开DFT

DFT/SCAN 的管控行为如下：

| LCS      | UNDEF | TEST | DEV | MANU | USER  | DEBUG | DEST |
|----------|-------|------|-----|------|-------|-------|------|
| dft_en 值 | OFF   | ON   | ON  | ON   | SW_ON | ON    | OFF  |

|            |     |    |    |    |     |    |     |
|------------|-----|----|----|----|-----|----|-----|
| dft_en 默认值 | OFF | ON | ON | ON | OFF | ON | OFF |
|------------|-----|----|----|----|-----|----|-----|

## 8.4 密钥管理

密钥生成与安全存储

- 支持密钥管理，包括：密钥的导入，导出，更新，使用，存储，派生
- 支持密钥更新、注销，轮换

### 8.4.1 密钥的存储

| Table 37: eHSM Key |       |             |          |            |                                  |
|--------------------|-------|-------------|----------|------------|----------------------------------|
| Key Name           | Level | Owner       | Location | KMU Key ID | Description                      |
| Chip RTL Key       | –     | Chip        | RTL      | –          | 仅在启动期间由硬件使用                      |
| Chip RTL Kek eHSM  | –     | Chip        | RTL      | –          | 用于安装 eHSM 密钥                     |
| Chip RTL Kek SoC   | –     | Chip        | RTL      | –          | 用于安装 SoC 密钥                      |
| Chip Root Key      | 0     | Chip        | OTP-KMU  | 0          | 仅在启动期间由硬件使用                      |
| Device Root Key    | 1     | Device      | OTP-KMU  | 1          | 仅在启动期间由硬件使用                      |
| OTP Key 2          | 1     | Chip        | OTP-KMU  | 2          | 用户根密钥，如可用于在导入（或导出）用户密钥时进行解密（或加密） |
| OTP Key N          | 1/2   | Chip/Device | OTP-KMU  | FW mapping | 由软件定义                            |

如上表所示：

密钥的存储分为3个地方，分别是：

代码块

```
1  RTL: Chip RTL Key /Chip RTL Kek eHSM/Chip RTL Kek SOC
2  eFUSE :Chip root key/Device Root /OTP key ...
3  KMU: Runtime Key Storage, 存储在 efuse中key, 被解密存储到 KMU
```

上述存储区域均位于eHSM安全边界内。

除eHSM外，C908，调试接口以及其它Master均无法直接访问OTP、KKMU中的密钥内容。

RTL key存储于rtl内部，可用于软件在烧写otp 的key时加密传输。

密钥的层级参考上表所示：

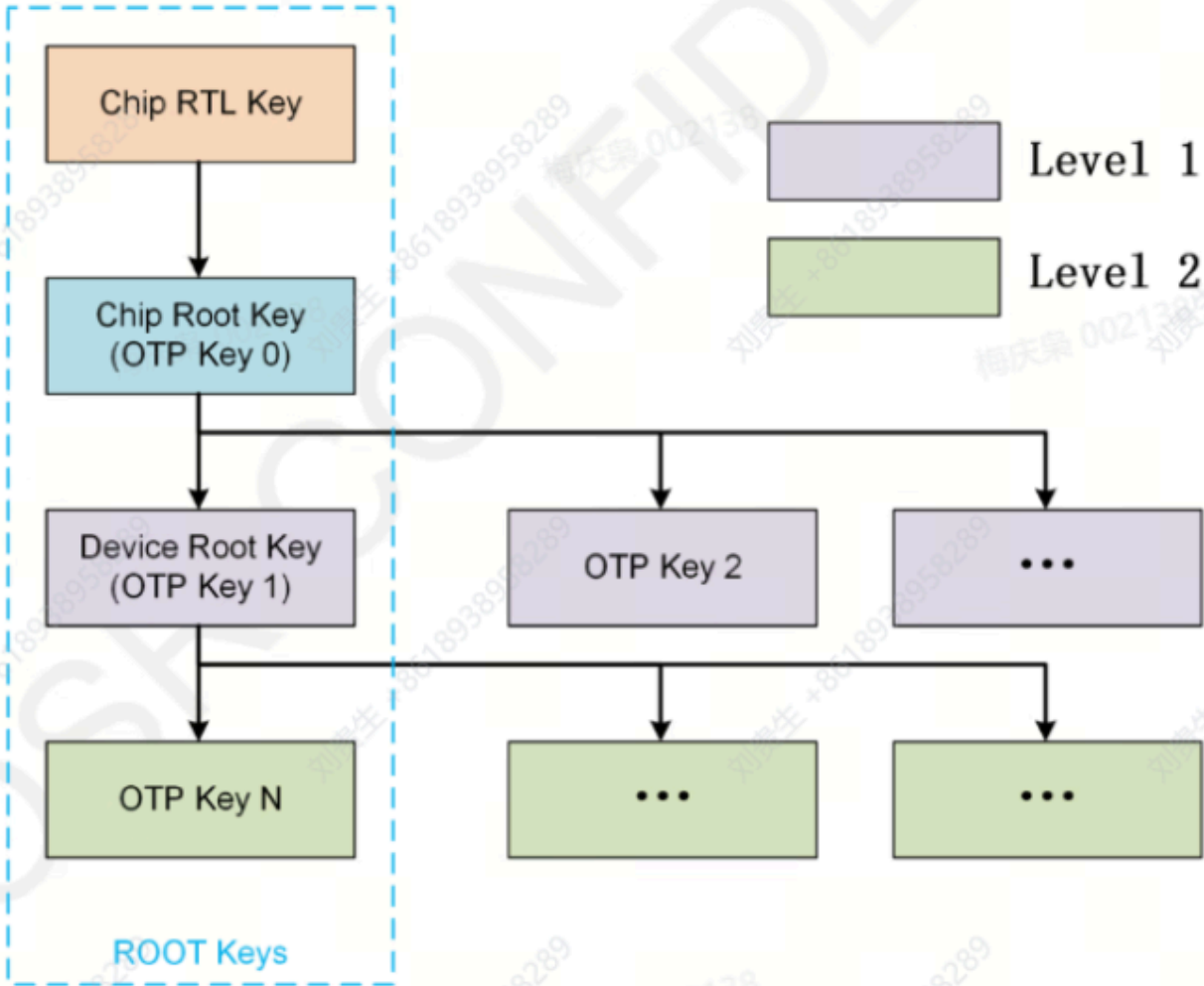
密钥级别

- 0: 由” Chip RTL Key” 加密
- 1: 由” Chip ROOT Key” 加密

- 2: 由” Device ROOT Key” 加密
- 密钥映射：eHSM 固件支持密钥的映射，密钥的 ID 取决于固件

8.4.2 密钥的安装和使用

Figure 18: eHSM 密钥层级



如上图所示，蓝色虚线内的key为根密钥

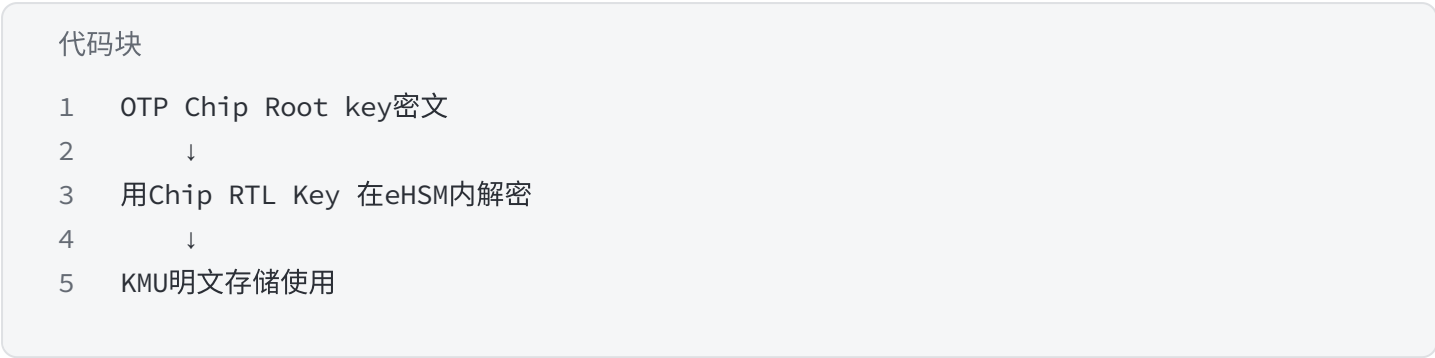
8.4.2.1 Jtag通路安装密钥：

1. Chip RTL Key的使用：

Chip RTL Key 可用于Chip root key/device root key 的安装。以Chip root key为例：

```
代码块
1  KMS/HSM 生成随机数作为Chip Root key
2  ↓
3  使用Chip RTL Key-加密
4  ↓
5  写入OTP
```

当Chip Root key被使用时：

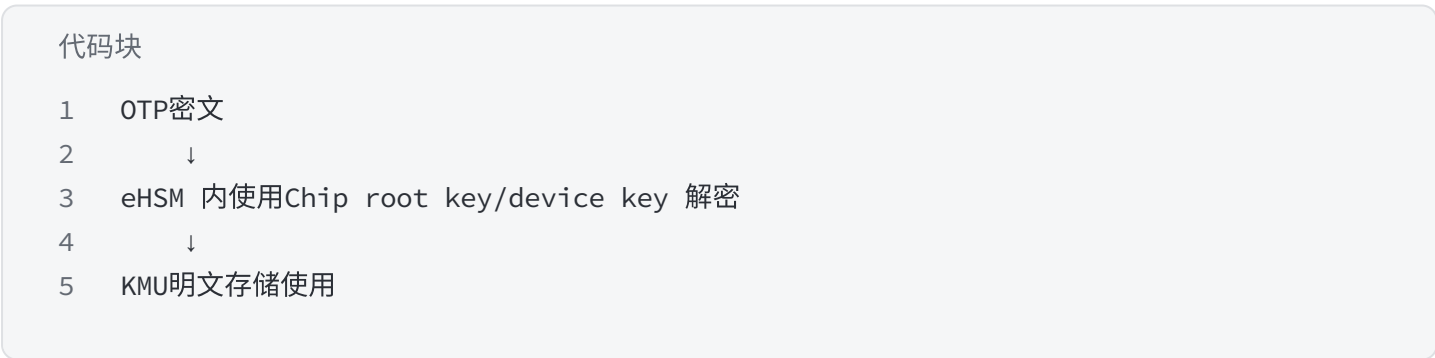


2. Chip RTL KEK eHSM/SOC 的使用：

Chip RTL KEK eHSM用于eHSM密钥安装。



当eHSM密钥被使用时：



整个过程均在eHSM内部完成。

明文密钥不会出现在SoC总线或外部存储器中。

Chip RTL KEK\_SoC同Chip RTL KEK eHSM的使用方式。

Chip RTL KEK SoC用于SoC业务密钥安装。

8.4.2.2 调用eHSM 安装密钥：

efuse中密钥的安装可参考：

## 密钥的撤销，更新和轮换

1. OTP/eFUSE中有关于密钥的生命周期管理的比特信息
2. OTP/eFUSE中计划放4组密钥，当前一组密钥泄密或者失效后，切换到下一组，同时更新对应的密钥生命周期管理信息
3. 对于处于”销毁”阶段的密钥，eHSM是不能读取使用密钥的
4. 软件的版本中有指示使用的密钥ID信息（和密钥组对应），eHSM调取对应分组的密钥即可

### 8.4.3 密钥派生机制

支持硬件级 KDF：

- AES-CMAC-KDF
- SHA-based KDF
- SM4-CMAC-KDF

派生原则：

- Key derivation input 绑定：
  - Device ID
  - LCS状态
  - Function domain
  - Nonce / challenge
- 派生过程不可逆
- 派生密钥默认 non-exportable

代码块

```
1  Root Key + Context (LCS + ID + Domain)
2          ↓
3  Hardware KDF (eHSM)
4          ↓
5  Derived Key (locked)
```

### 8.4.4 密钥的权限与访问控制

密钥使用权限由 eHSM + 生命周期共同控制：

控制规则：

- efuse和KMU中密钥无外部访问接口

- 调试接口不能访问密钥
- C908仅发起请求，不参与运算
- 密钥的使用和派生均在eHSM内部完成
- 其中Rtl key 软件可明文使用

密钥受生命周期管控，管控规则如下：

1. 当满足如下对应条件，密钥将被解密到 KMU
2. KMU密钥访问权限
3. DEST 阶段，软件需手动清除密钥

8.4.5 Otp 存储密钥内容（12个key）

|                     | 等级 | key 类型 | 位置       | 灌装 阶段     | Key from                                          | 备注          |
|---------------------|----|--------|----------|-----------|---------------------------------------------------|-------------|
| Chip root key       | 0  | sym m  | OTP- KMU | TEST /DEV | 1. ehsm：随机数<br>2. kms：key (chip rtl key 加密传输)     | 芯片根密钥       |
| Device root key     | 1  | sym m  | OTP- KMU | MAU N     | 1. ehsm：随机数<br>2. kms：key (chip rtl key 加密传输)     | 设备根密钥       |
| Secret key 0 (todo) | 2  | sym m  | OTP- KMU | MAU N     | 1. ehsm：随机数                                       |             |
| eHSM debug key      | 2  | asy mm | OTP- KMU | TEST /DEV | kms：公钥hash值 (chip root key 加密存储)                  | eHSM 鉴权验签密钥 |
| eHSM FW verify key  | 2  | asy mm | OTP- KMU | TEST /DEV | kms：公钥hash值 (chip root key 加密存储)                  | eHSM 镜像验签密钥 |
| eHSM encrypt key    | 2  | sym m  | OTP- KMU | TEST /DEV | 1. ehsm：随机数<br>2. kms：key (Chip RTL Kek eHSM加密传输) | eHSM 镜像解密密钥 |

|                                 |          |                   |                    |                     |                                                     |                                   |
|---------------------------------|----------|-------------------|--------------------|---------------------|-----------------------------------------------------|-----------------------------------|
| <del>eHSM-private-key</del>     | 2        | <del>asymmm</del> | <del>OTP-KMU</del> | <del>TEST/DEV</del> |                                                     | 随机数                               |
| DICE root CA key                | 2        | asymmm            | OTP-KMU            | TEST/DEV            | kms: 公钥hash值<br>(chip root key 加密)                  | DICE root CA key, 用于追溯            |
| Device private Key              | 2        | asymmm            | OTP-KMU            | TEST/DEV            | ehsm: 随机数                                           | DICE 设备私钥, attestation key        |
| Soc debug key                   | 2        | asymmm            | OTP-KMU            | MUN A               | kms: 公钥hash值<br>(device root key 加密存储)              | soc 鉴权密钥                          |
| Soc FW verify key               | 2        | asymmm            | OTP-KMU            | MAN U               | kms: 公钥hash值<br>(device root key 加密存储)              | soc 镜像验签密钥                        |
| Soc encrypt key                 | 2        | symm              | OTP-KMU            | MAN U               | 1. ehsm: 随机数<br>2. kms: key (Chip RTL Kek eHSM加密传输) | soc 镜像解密密钥                        |
| UDS                             | 2        | asymmm            | OTP-KMU            | TEST/DEV            | ehsm: 随机数                                           | UDS                               |
| User auth key                   | 2        | asymmm            | OTP-KMU            | MAN U               | kms: 公钥hash值<br>(device root key 加密存储)              | 用于生命周期从USER MODE切换到DEBUG MODE时的鉴权 |
| <b>Soc debug key1 (待删除)</b>     | <b>2</b> | asymmm            | <b>OTP-KMU</b>     | <b>MAN U</b>        | kms: 公钥hash值<br>(device root key 加密存储)              | 密钥轮换key, soc 鉴权密钥                 |
| <b>Soc FW verify key1 (待删除)</b> | <b>2</b> | asymmm            | <b>OTP-KMU</b>     | <b>MAN U</b>        | kms: 公钥hash值<br>(device root key 加密存储)              | 密钥轮换key, soc 镜像验签密钥               |
| <b>Soc encrypt key1 (待删除)</b>   | <b>2</b> | symm              | <b>OTP-KMU</b>     | <b>MAN U</b>        | 1. ehsm: 随机数<br>2. kms: key (Chip RTL Kek SOC加密传输)  | 密钥轮换key, soc 镜像解密密钥               |
|                                 | <b>2</b> |                   |                    |                     | 外部                                                  | 密钥轮换key                           |

|                    |   |  |         |       |      |         |
|--------------------|---|--|---------|-------|------|---------|
| Soc-debug-key2     |   |  | OTP-KMU | MAN U |      |         |
| Soc-FW-verify-key2 | 2 |  | OTP-KMU | MAN U | 外部   | 密钥轮换key |
| Soc-encrypt-key2   | 2 |  | OTP-KMU | MAN U | ehsm | 密钥轮换key |
| Soc-debug-key3     | 2 |  | OTP-KMU | MAN U | 外部   | 密钥轮换key |
| Soc-FW-verify-key3 | 2 |  | OTP-KMU | MAN U | 外部   | 密钥轮换key |
| Soc-encrypt-key3   | 2 |  | OTP-KMU | MAN U | ehsm | 密钥轮换key |

## 8.5 加解密服务

eHSM提供一系列对称加解密&hash&非对称算法服务。具体支持的算法和模式详见vendor 文档：《eHSM HP数据》

**支持对称算法：**

1. AES-128/AES-192/AES-256/SM4/DES/TDES算法
2. CBC/CFB/OFB/CTR/CMAC/CBC-MAC/GCM/CCM/GMAC/XTS工作模式

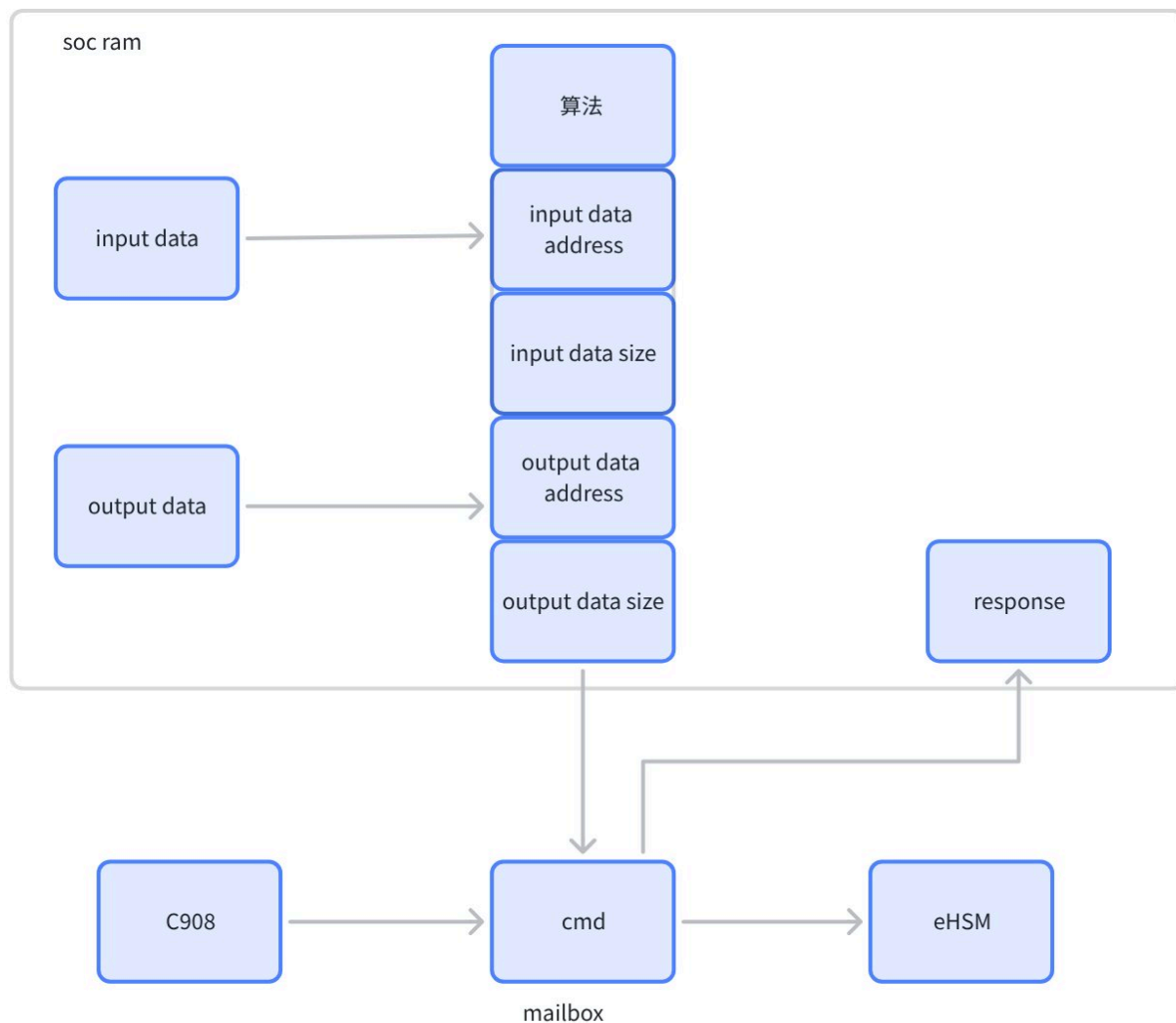
**支持非对称算法：**

1. secp256/secp384/secp521/SM2/Ed25519/brainpoolp椭圆曲线加密和数字签名的硬件支持能力
2. 支持RSA1024/2048/3072/4096，SM2

**支持摘要算法：**

1. SHA256/SHA384/SHA512/SHA3-256/SHA3-384/SHA3-512/SM3
2. 支持HMAC





管理子系统C908与 eHSM 之间通过 Mailbox 进行命令和响应的交互，通过 SOC\_MEM进行数据的传递。SOC 系统与 eHSM 交互过程如下：

1. SOC CPU 将待处理的数据存入 SOC\_MEM 中。
2. SOC CPU 通过 Mailbox 将待处理数据的地址、处理结果输出地址、所需操作模式(如加解密)等信息通过command 的形式发送给 eHSM，并触发 eHSM 中断。
3. eHSM 接收并解析 command 命令，从 SOC\_MEM 取出数据并进行处理。
4. eHSM 处理完成，将结果放置在soc mem中指定的 SOC\_MEM 相应地址中。
5. eHSM 发送 Response 置起 Mailbox 中的完成标识，SOC CPU 通过轮询/中断方式获知eHSM 处理完成标识。
6. SOC CPU 从 SOC\_MEM 相应地址中取出结果。

## 8.6 状态错误信息

1. eHSM提供一组输出接口，用于指示eHSM内部发生的error和status信息：o\_hsm\_status[63:0]/o\_hsm\_err\_hw[63:0]/o\_hsm\_err\_fw[63:0]，SOC需做逻辑记录error和status信息

2. eHSM提供一组输入接口，用于接收SOC发生的error和status信息：  
i\_soc\_status[31:0]/i\_soc\_err[31:0]，当前项目没有用途。

### 8.6.1 硬件上报错误&状态管理

1. eHSM提供一组输出接口，o\_hsm\_status[63:0] /o\_hsm\_err\_hw[63:0]用于指示eHSM内部发生的error和status信息
2. eHSM提供一组输入接口，用于接收SOC发生的error和status信息：  
i\_soc\_status[31:0]/i\_soc\_err[31:0]，当前没有用途。
3. o\_hsm\_err\_hw[63:0]包含电平和脉冲信号，可参考vendor技术文档

下表为vendor 提供的error上报全集，其中当前版本eHSM可使用的o\_hsm\_err\_hw[63:0]功能为：

1. 提供**misc error**上报
2. **Memory ecc Error**上报

Table 11: o\_hsm\_err\_hw 释义

| Bits                      | Field                  | Type  | Description                                    |
|---------------------------|------------------------|-------|------------------------------------------------|
| <b>DMAC Timeout Error</b> |                        |       |                                                |
| 62:61                     | dma_ahb_rbus/wbus_to   | Level | DMA AHB 读总线（写总线）超时，每条总线对应 1 位                  |
| 60:59                     | dma_axi_rabus/rdbus_to | Level | DMA AXI 读地址（读数据）总线超时，每条总线对应 1 位                |
| 58:57                     | dma_axi_wabus/wdbus_to | Level | DMA AXI 写地址（写数据）总线超时，每条总线对应 1 位                |
| <b>Bus Parity Error</b>   |                        |       |                                                |
| 55                        | socprc_pchk_err        | Level | SOC 总线奇偶校验错误                                   |
| 54                        | nvmpc_pchk_err         | Level | NVM 总线奇偶校验错误                                   |
| 53                        | ahbprc_pchk_err        | Level | AHB 总线奇偶校验错误                                   |
| 52                        | dramprc_pchk_err       | Level | DRAM 总线奇偶校验错误                                  |
| 51                        | iramprc_pchk_err       | Level | IRAM 总线奇偶校验错误                                  |
| 50                        | iromprc_pchk_err       | Level | IROM 总线奇偶校验错误                                  |
| 49                        | ahbdpre_pchk_err       | Level | AHB_DMA 总线奇偶校验错误                               |
| 48                        | spre_pchk_err          | Level | Sbus 奇偶校验错误                                    |
| 47                        | dpre_pchk_err          | Level | Dbus 奇偶校验错误                                    |
| 46                        | ipre_pchk_err          | Level | Ibus 奇偶校验错误                                    |
| <b>Misc Error</b>         |                        |       |                                                |
| 40                        | wdt_timeout            | Level | 看门狗计时器超时                                       |
| 35                        | otp_key_crc_err        | Level | OTP 密钥 CRC 错误                                  |
| 34                        | hw_trng_retry_warning  | Level | 启动期间 TRNG 健康检测失败。                              |
| 33                        | hw_trng_retry_fail     | Level | 启动期间 TRNG 健康检测重启超时。                            |
| 32                        | hw_trng_ht_fail        | Level | TRNG 健康测试失败（在启动和功能阶段）                          |
| <b>SoC Bus Error</b>      |                        |       |                                                |
| 29                        | soc_err_ahb_cfg        | Level | CFG AHB 总线响应错误                                 |
| 28                        | soc_err_ahb_nvm        | Level | NVM AHB 总线响应错误                                 |
| 27                        | soc_err_ahb_otp        | Level | OTP AHB 总线响应错误                                 |
| 26                        | soc_err_ahb_mem        | Level | SoC Memory AHB 总线响应错误                          |
| 25                        | soc_err_axi_dma_rd     | Level | DMA AXI 总线读通道响应错误                              |
| 24                        | soc_err_axi_dma_wr     | Level | DMA AXI 总线写通道响应错误                              |
| <b>Memory ECC Error</b>   |                        |       |                                                |
| 23:20                     | reserved               | —     | —                                              |
| 19:16                     | mem_ecc_mb_pke3/2/1/0  | Level | PKE3/PKE2/PKE1/PKE0 RAM 不可纠正错误，每个相应 RAM 各有 1 位 |
| 15                        | mem_ecc_mb_kmu         | Level | KMU RAM 不可纠正错误                                 |
| 14                        | mem_ecc_mb_dram        | Level | DRAM 不可纠正错误                                    |
| 13                        | mem_ecc_mb_iram        | Level | IRAM 不可纠正错误                                    |
| 12                        | mem_ecc_mb_irom        | Level | IROM 不可纠正错误                                    |
| 11:8                      | reserved               | —     | —                                              |
| 7:4                       | mem_ecc_1b_pke3/2/1/0  | Level | PKE3/PKE2/PKE1/PKE0 RAM 可纠正错误，每个相应 RAM 各有 1 位。 |
| 3                         | mem_ecc_1b_kmu         | Level | KMU RAM 可纠正错误                                  |
| 2                         | mem_ecc_1b_dram        | Level | DRAM 可纠正错误                                     |
| 1                         | mem_ecc_1b_iram        | Level | IRAM 可纠正错误                                     |
| 0                         | mem_ecc_1b_irom        | Level | IROM 可纠正错误                                     |

- 以上信息需要记录到sec\_cfg寄存器里，供软件查询。
- 脉冲信号，SOC在记录时应对脉冲做展平处理，并对原始脉冲信号进行计数。

- 电平信号，SOC可通RO寄存器模块存储err/status电平信息
- o\_hsm\_err\_hw[63:0]中的error电平信号，同时会上报到eHSM EMU，会hold住ehsm CPU，eHSM无法处理，只能通过复位eHSM恢复。

硬件o\_hsm\_err\_hw[63:0]发生上表中的错误，处理方式为：

表 1 硬件严重错误

| 错误位   | 描述                          |
|-------|-----------------------------|
| 62:57 | DMAC Timeout Error          |
| 55:46 | Bus Parity Error            |
| 40    | 看门狗计时器超时                    |
| 35    | OTP 密钥 CRC 错误               |
| 29:24 | SOC Bus Error               |
| 19:11 | Memory ECC Multi-Bits Error |

1. 需要复位：发生表1中错误为严重错误，需要复位eHSM
2. 无需复位：其它硬件错误 Firmware 会自动处理或忽略，不影响 Firmware 继续进行服务，但硬件仍然会通过 EMU 直接上报 SOC

下表为o\_hsm\_status[63:0]状态信息：

Table 10: o\_hsm\_status 释义

| Bits  | Field           | Description         |
|-------|-----------------|---------------------|
| 31:28 | <i>reserved</i> | —                   |
| 27    | cpu_hart_halted | SYS_HSM_STA1[27]    |
| 26    | <i>reserved</i> | —                   |
| 25    | cpu_wfi         | SYS_HSM_STA1[25]    |
| 24:21 | hsm_fw_sta1     | SYS_HSM_STA1[24:21] |
| 20    | soc_cpu_release | SYS_HSM_STA1[20]    |
| 19    | soc_cpu_reset   | SYS_HSM_STA1[19]    |
| 18    | soc_reset       | SYS_HSM_STA1[18]    |
| 17    | soc_dbg_en      | SYS_HSM_STA1[17]    |
| 16    | hsm_dbg_en      | SYS_HSM_STA1[16]    |
| 15    | hsm_fw_sta0     | SYS_HSM_STA0[31]    |
| 14    | hsm_lc_undef    | SYS_HSM_STA0[30]    |
| 13    | hsm_lc_destroy  | SYS_HSM_STA0[29]    |
| 12    | hsm_lc_debug    | SYS_HSM_STA0[28]    |
| 11    | hsm_lc_user     | SYS_HSM_STA0[27]    |
| 10    | hsm_lc_manu     | SYS_HSM_STA0[26]    |
| 9     | hsm_lc_dev      | SYS_HSM_STA0[25]    |
| 8     | hsm_lc_test     | SYS_HSM_STA0[24]    |
| 7     | soc_verify_err  | SYS_HSM_STA0[23]    |
| 6     | soc_verify_done | SYS_HSM_STA0[22]    |
| 5     | firmware_err    | SYS_HSM_STA0[21]    |
| 4     | firmware_done   | SYS_HSM_STA0[20]    |
| 3     | bootloader_err  | SYS_HSM_STA0[19]    |
| 2     | bootloader_done | SYS_HSM_STA0[18]    |
| 1     | hw_boot_err     | SYS_HSM_STA0[17]    |
| 0     | hw_boot_done    | SYS_HSM_STA0[16]    |

### 8.6.2 软件上报错误&异常信息

通过o\_hsm\_err\_fw[63:0]可将软件异常传到SOC。eHSM内部通过EMU处理异常信息。

#### 1. Bootloader 上报软件错误参考下表：

表 2 软件错误信号与 BL 行为

| SW Error Code         | ERR_FW bit | 说明                                  | BL 处理方式 |
|-----------------------|------------|-------------------------------------|---------|
| secboot_selftest_fail | 0          | 自检时有算法失败                            | 3       |
| selftest_hash_fail    | 16         | HASH 自检失败                           | 3       |
| selftest_ske_fail     | 17         | SKE 自检失败                            | 3       |
| selftest_pke_fail     | 18         | PKE 自检失败                            | 3       |
| selftest_trng_fail    | 19         | 随机数自检失败                             | 3       |
| wdg_timeout           | 37         | Watchdog 超时，此位由软件设置                 | 3       |
| bl_verify_failed      | 40         | BootLoader ROM code 校验失败，ROM 可能已损坏  | 3       |
| cpu_exception         | 41         | CPU 异常                              | 3       |
| otp_init_failed       | 43         | 调用 otp_init 驱动初始化 OTP 失败            | 3       |
| uid_crc32_mismatch    | 62         | OTP 中 UID 的 CRC32 值不正确，非 TEST 模式会检查 | 1       |
| hw_version_mismatch   | 63         | 软件与硬件的版本或客户编号不匹配                    | 1       |

表 1 BL 错误处理定义

| 代号 | 处理方式                                                                                                                                                                                                                                           |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 忽略错误，继续运行                                                                                                                                                                                                                                      |
| 2  | <ul style="list-style-type: none"><li>如果 BootLoader 处于启动阶段，则拉起bootloader_err和bootloader_done信号，然后 Hold 住 CPU 不再执行 (while (1))</li><li>如果 BootLoader 处于命令处理阶段(即bootloader_done已拉起)，则 Hold 住 CPU 不再执行 (while (1)) 且不会拉起bootloader_err)</li></ul> |

注：

- 软件错误信号的低 48 位作为错误位，发生错误后软件会 Hold CPU 不再继续运行
- 软件错误信号的高 16 位作为警告位，发出警告后软件继续运行
- 软件信号是电平信号，软件不会清除对应信号
- 在生命周期 TEST MODE，软件会忽略所有错误，尝试继续运行

Hold 住CPU时，需要复位eHSM。

Hold 住CPU时，需要复位eHSM。

2. Firmware 上报软件错误参考下表：



表 2 软件错误

| 错误位   | 错误描述                           | 处理方式 |
|-------|--------------------------------|------|
| 47    | 检测到错误注入攻击                      | 挂起   |
| 46    | 栈保护检查失败                        | 挂起   |
| 45    | 硬件寄存器写入失败                      | 挂起   |
| 37    | WDT 中断触发 <sup>2</sup>          | 挂起   |
| 35    | UART 调试鉴权协议错误                  | 继续执行 |
| 34    | Mailbox 调试鉴权协议错误               | 继续执行 |
| 33    | 清除 s2h_note 寄存器未成功             | 继续执行 |
| 32    | Mailbox 中断触发但 s2h_note 寄存器未置 1 | 继续执行 |
| 31:28 | CPU 异常 <sup>3</sup>            | 挂起   |

1. 警告：仅上报 SOC，Firmware 继续执行
2. 错误：除了上报 SOC，Fireware 会挂起并拒绝继续服务

## 8.7 中断

1. eHSM支持mailbox中断

Table 31: Mailbox 中断接口

| Signal        | Direction | Width | Description       |
|---------------|-----------|-------|-------------------|
| o_mailbox_irq | Out       | 1-16  | eHSM mailbox 中断请求 |

2. 支持eHSM Watchdog timeout中断
3. 支持 eHSM memory ecc 多bit 错误 中断
4. 支持 eHSM 上报OTP 密钥ECC 错误
5. 支持 eHSM 上报 o\_hsm\_err\_hw[63:0] 中除Watchdog超时/memory ecc 多bit 错误/OTP 密钥ECC 错误 外的非严重错误
6. 支持efuse上报异常中断

## 8.8 测试接口

### 8.8.1 随机数测试

ehsm提供一组随机数测试接口，该调试接口需要接到PAD口上，用于测试随机数的熵源是否正常工作。

- 随机数测试在LCS=TEST模式下进行
- 随机数测试需将测试信号o\_trng\_ro\_clk/o\_trng\_ro\_out输出到PAD口进行测试，可选在PAD口观测所有8bit信号，也可在PAD资源紧张时，观测部分信号或者分时观测8bit信号。
- 测试信号为慢环的32分频，假设慢环为100M，典型值=100/32=3MHz，典型值可用于自行review随机数是否正常工作。
- 随机数应在后仿时带反标的网表上观测起震情况，观测o\_trng\_ro\_clk是否有较明显相位差。
- o\_trng\_rdy/o\_trng\_alarm悬空，为eHSM内部使用信号。

### 8.8.2 UART调试

ehsm提供一UART调试接口，复用接口。该调试接口需要接到PAD口上，用于打印调试信息。

## 9. DICE/SPDM方案

DICE和attestation是保障系统完整性和可信度的重要手段。DICE通过建立一个动态的，可验证的完整性链，对系统的各个组件进行监测和验证。attestation则是通过向外部实体提供系统的完整性证明，确保系统运行状态和配置符合预期。这两个功能结合起来，能够有效的防止系统被篡改和攻击。

### 9.1 DICE

DICE通过编码层特定的测量值（CDI）来确保设备的每一层都符合预期的安全标准。

方案一：DICE证书链

首先：HSM 的 rom 里面，在首次开机的时候去检测UDS的数据，检测 OTP 里面有没有数据。如没有数据，使用TRNG去生成一个random数据作为随机数，软件根据random数据生成UDS。

C908 执行 secure boot时，同时会触发到DICE的一次计算。

DICE的计算流程：

前置条件：在 eHSM 的 firmware 中运行，生成UDS

step1

- 在ehsm ROM中解析eHSM FM code
- 使用解析出的HSMFM code 和UDS进行一个密钥派生，生成CDI1，使用CDI1派生出一对公钥的key
- 使用UDS 派生出一对Device key，将public key上传到安全服务器，UDS派生出的private key对CDI1派生后的public key进行签名



- 生成证书device\_cert

#### step2

- 解析需要度量的镜像，获取下一级firmware的代码
- 对获取到的firmware 的进行哈希
- hash值跟上级的CDI1进行一个hmac，然后生成一个新的CDI2，使用CDI2派生出一对公钥key
- 使用CDI1派生出的private key对CDI2派生后的public key进行签名
- 生成证书L1 cert

#### step3

- boot到下一级firmware
- 对获取到的下一级firmware code做哈希，获取到hash值
- 以上 hash值跟CDI2进行一个hamc，然后生成一个新的CDI3，使用CDI3派生出一对公钥key
- 使用CDI2派生出的private key对CDI3派生后的public key进行签名
- 生成证书L2 cert

#### step4

- 如有下一级需要校验的镜像，重复以上步骤3

证书验证路径 Root Cert---->Device Cert ---> L0 Cert ---> L1 Cert ---> L2 Cert

### 方案二：静态链+度量值

#### 静态信任链

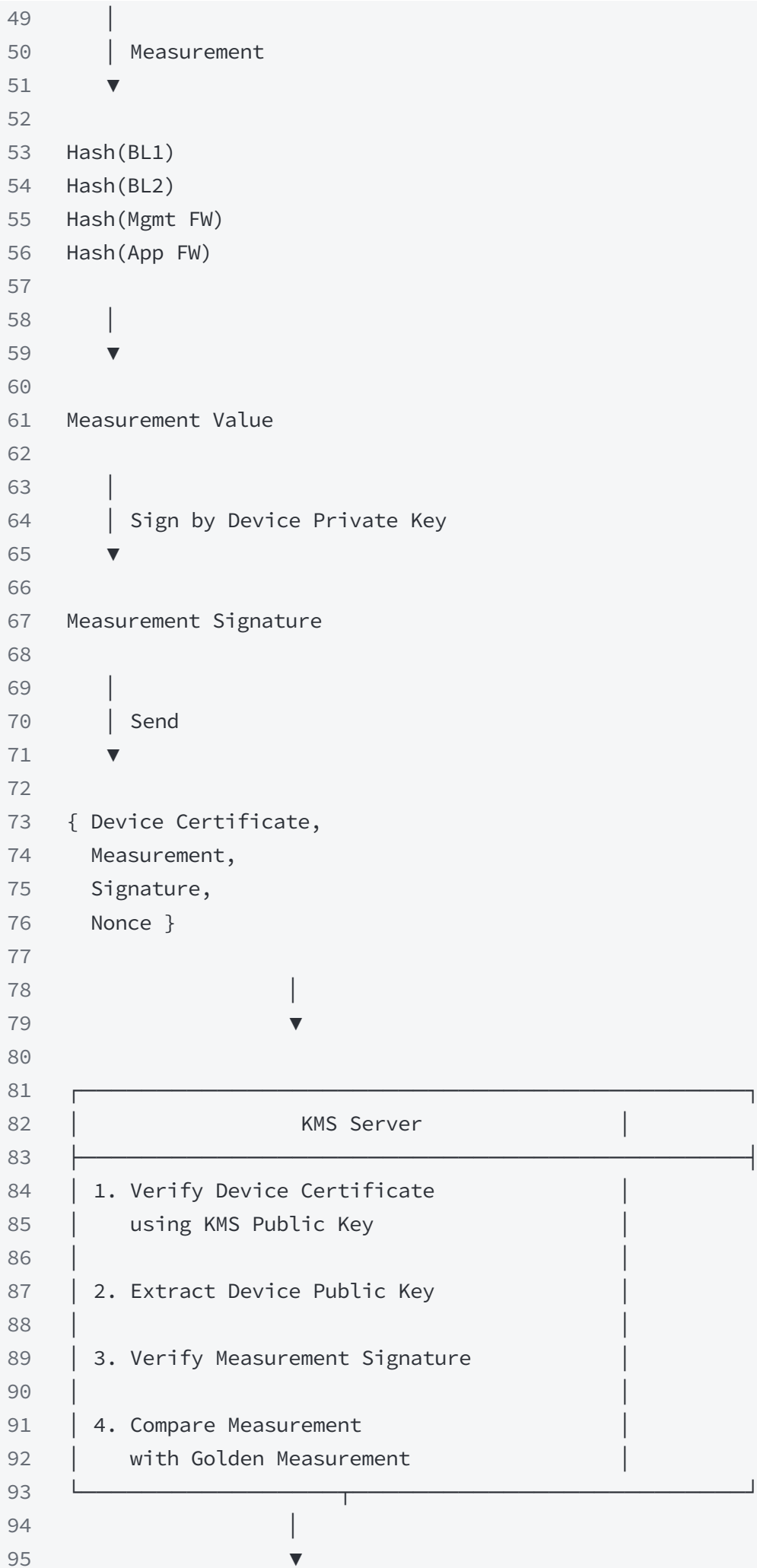
- KMS服务器用CA私钥签发设备证书（X.509），证书中包含设备公钥。
- 芯片eFuse中存储设备私钥（Device private Key），对应公钥提前注册到KMS服务器。

#### 验证流程

- 启动阶段：芯片用预置Root CA公钥验证服务器下发的设备证书。
- 度量阶段：芯片对关键数据（BootROM、固件等）计算Hash/度量值（Measurement），然后用设备私钥签名度量值，而不是签名镜像本身。
- 远程验证阶段：服务器使用设备公钥验证芯片发送的度量值签名，确认设备身份和运行环境的完整性。

代码块

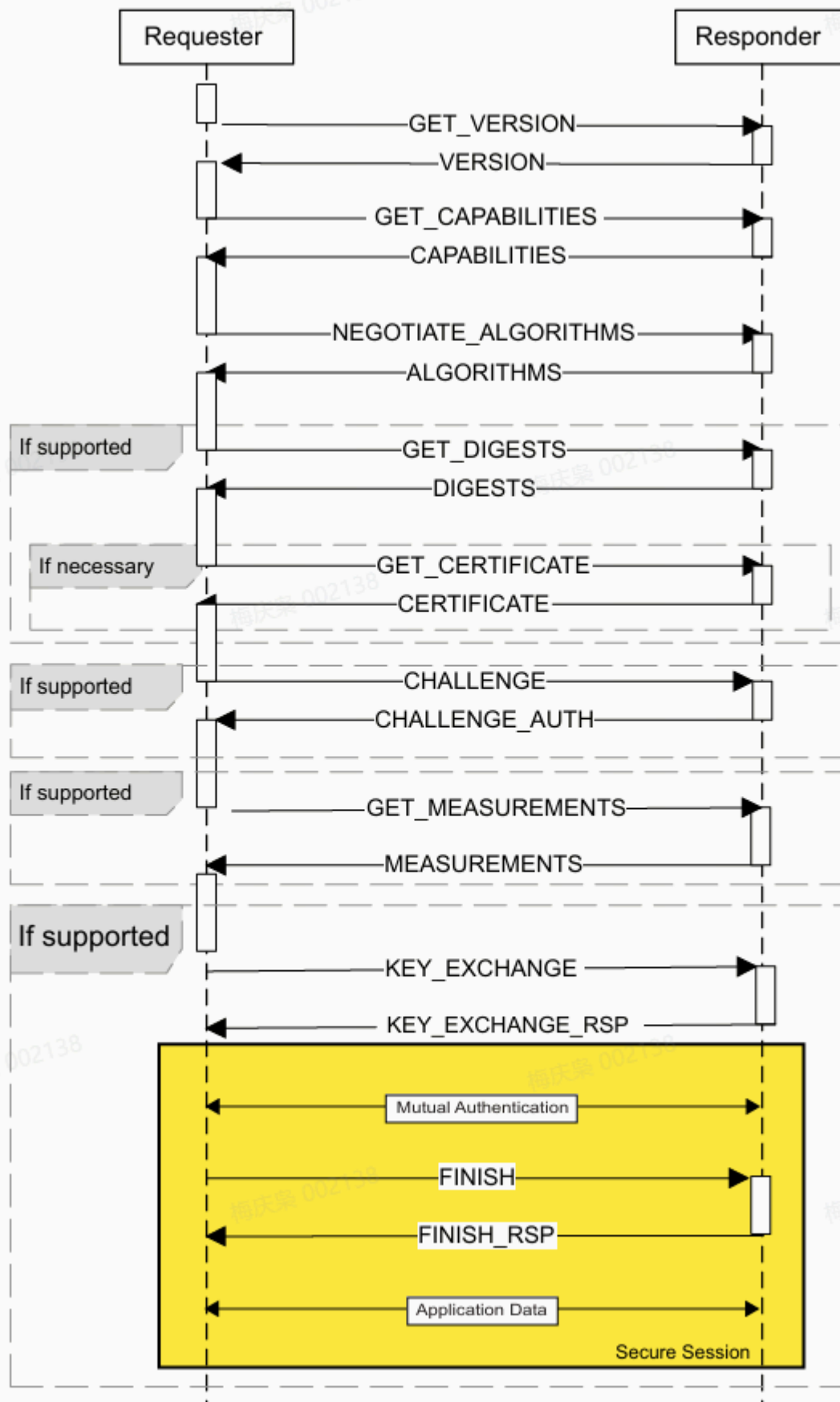




## 9.2 SPDM

SPDM是一种安全通信协议，负责在设备和管理系统之间传输测量数据。由软件实现SPDM流程建议支持：

- SPDM 1.2
- 安全Session
- Runtime认证



## 10. firewall

### 10.1 Userid说明

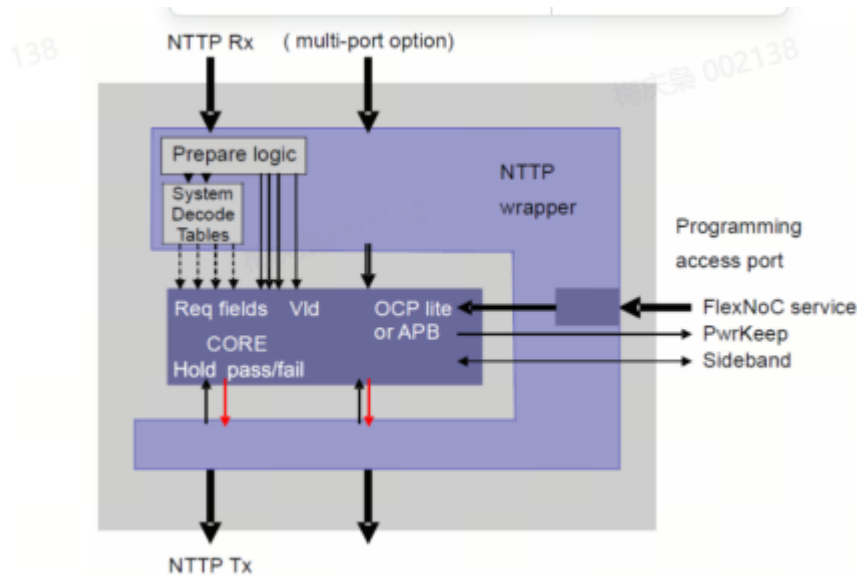
Userid的使用说明

- 通过Userid实现用户对IRAM和Slave模块的权限管理
- Userid代表Master分组或者权限，允许多个Master共用一个Userid，也可以一个master一个Userid
- Userid共4bit，表示有16组用户，每组用户对某个区域或者Slave模块的读写权限可分开设置
- Die 0和Die 1 不共用id。

|       | USERID  | 适配Master对象                | 说明                  |
|-------|---------|---------------------------|---------------------|
| Die 0 | 4'b0000 | eHSM                      |                     |
|       | 4'b0001 | 管理子系统RAS Core             |                     |
|       | 4'b0010 | 管理子系统中的通用&带内&带外管理MCU Core |                     |
|       | 4'b0011 | 管理子系统中的Codec MCU Core     |                     |
|       | 4'b0100 | 管理子系统中功耗管理MCU Core        |                     |
|       | 4'b0101 | Host通过PCIe访问管理子系统中的RAM    | 走config bus channel |
|       | 4'b0110 | 管理子系统中的DMA权限              |                     |
|       | 4'b0111 | TOP NOC中的DMA              |                     |
| Die 1 | 4'b1000 | eHSM                      |                     |
|       | 4'b1001 | 管理子系统RAS Core             |                     |
|       | 4'b1010 | 管理子系统中的通用&带内&带外管理MCU Core |                     |
|       | 4'b1011 | 管理子系统中的Codec MCU Core     |                     |
|       | 4'b1100 | 管理子系统中功耗管理MCU Core        |                     |
|       | 4'b1101 | Host通过PCIe访问管理子系统中的RAM    | 走config bus channel |
|       | 4'b1110 | 管理子系统中的DMA权限              |                     |
|       | 4'b1111 | TOP NOC中的DMA              |                     |
|       | 其他      | reserved                  |                     |
|       |         |                           |                     |

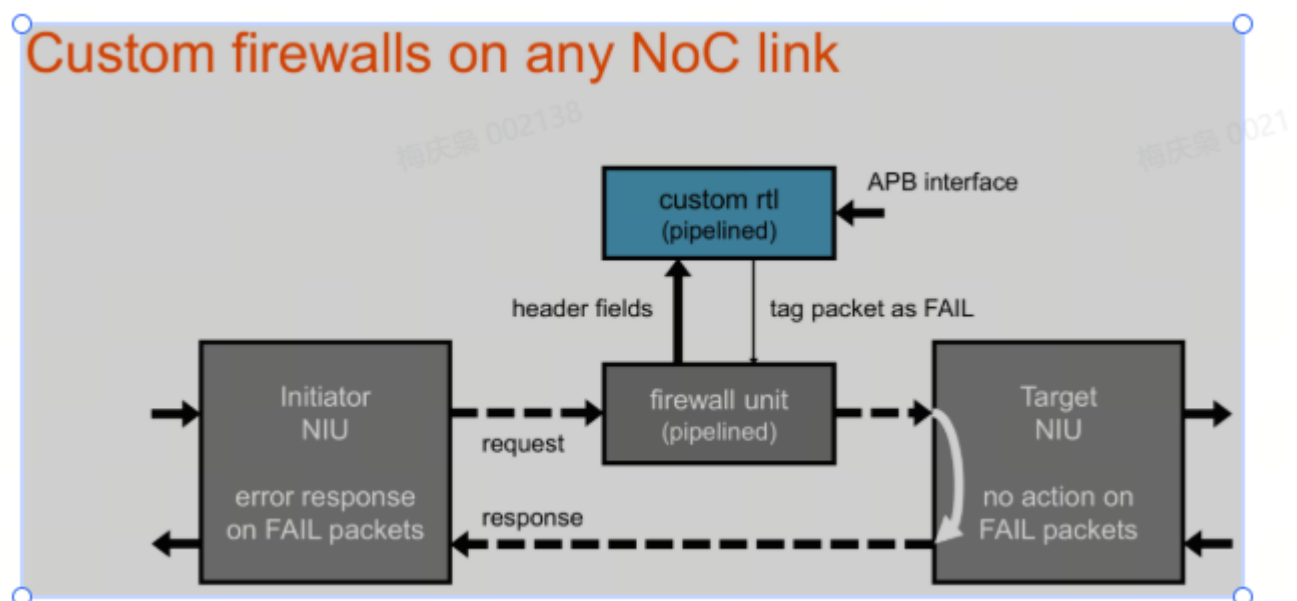
10.2 Firewall功能

Firewall是NOC的一个组件，可以放到任何一个Initiator NIU到Target NIU的链路上，一般是靠近Target NIU放置。



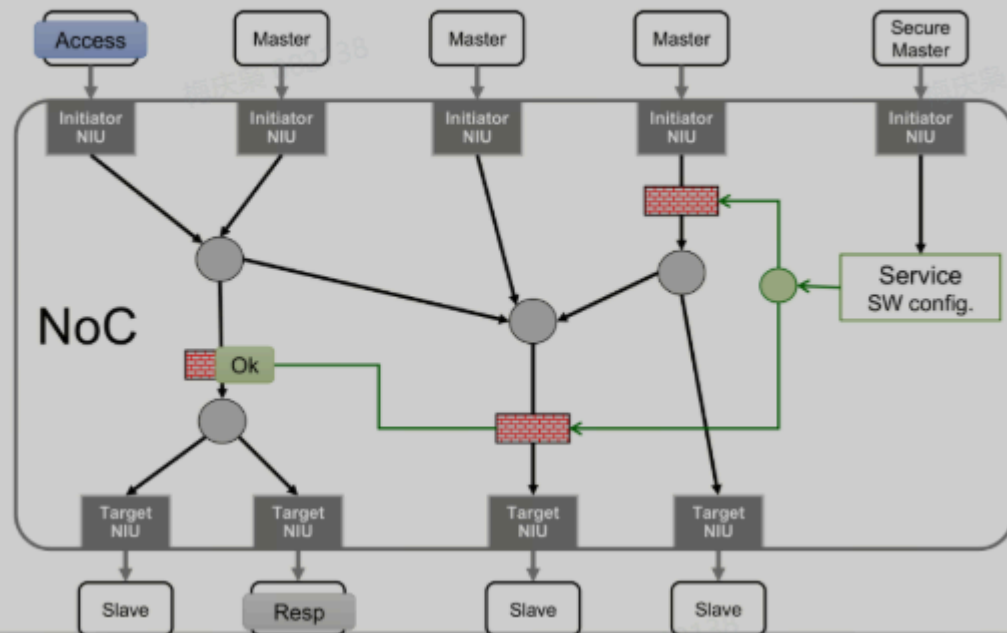
Firewall由WRAPPER和用户逻辑两部分构成，并通过特定的接口进行信息传递。传递给Target NIU的每一次传输，WRAPPER会提取包头信息给用户逻辑，用户逻辑根据配置寄存器，对传输的权限进行判断，并将判断结果和隐藏指示反馈给WRAPPER，针对反馈结果NOC会做出如下操作

1. 对于check pass的传输, Target NIU会将其透传给SLAVE，并由SLAVE做出 Response。
2. 对于check fail的传输，Target NIU不会将其传给SLAVE
3. 当不隐藏错误时，由Target NIU产生C；
4. 当需要隐藏错误时，由Initiator NIU 产生Response OK, 此时Initiator是察觉不到访问错误的，这样就可以防止一些恶意攻击。



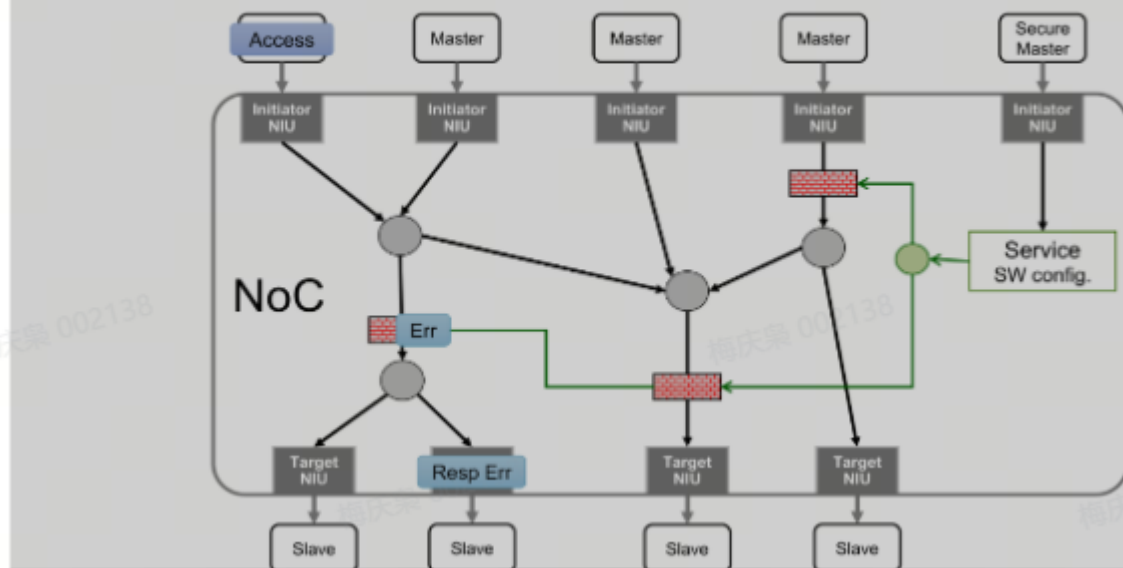
图： Custom firewalls

## Security firewall – Security passed



图：Security passed

## Security firewall – Security Error



图：Security Error



### Example – Firewall fail with HIDE error

The diagram illustrates a network-on-chip (NoC) architecture where a firewall failure occurs. At the top, five Master nodes are shown. The first Master is connected to an Access block, which sends a 'Resp OK' signal. The other four Masters are connected to Initiator NIU (Network Interface Unit) blocks. The NoC is represented by a central area with nodes and connections. A Service SW config. block is connected to the NoC. Below the NoC, there are four Target NIU blocks, each connected to a Slave node. A Hide block is connected to the NoC and sends a 'Resp Hide' signal. A red box labeled 'Firewall' is shown in the NoC area, indicating a failure point. A green arrow points from the Service SW config. block to the Firewall box. The text 'NoC' is prominently displayed in the center of the diagram.

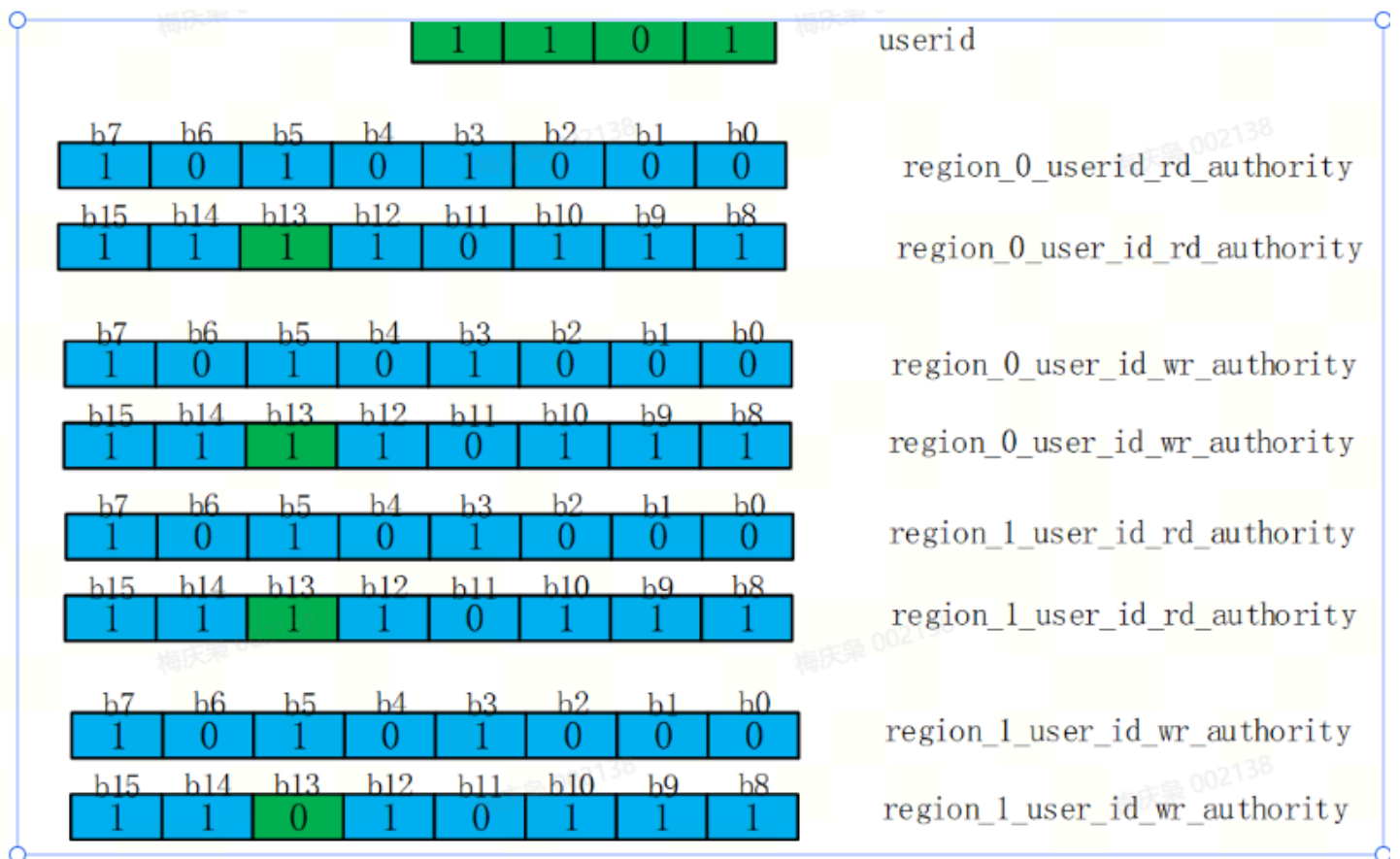
图: HIDE error

## Firewall的用户逻辑的设计:

1. 用户逻辑在check使能的情况下，对传输做检查，否则透传。
2. 在check使能的情况下，用户逻辑会进行地址检查，安全检查和用户ID访问权限检查。只要**有一个检查是fail的，那结果就是fail的**。另外在隐藏错误使能的情况下，会送出隐藏指示信号。
3. 用户逻辑会将slave划分成若干个区间。
  - 每个区间由起始地址和结束地址划定范围。
  - 每个区间的读安全属性和写安全属性可分别配置。
  - **每个区间的userid 读权限和userid 写权限可分别配置。**
  - **每个区间可以单独使能。只有使能的区间才会进行到检查，否则该区间会被忽略掉。**
  - 一个区间可以对应一个slave，如apb slave的情况。也可以多个区间对应一个slave，如slave是sram的情况，sram最多可以被划分成8个区间，每个区间的安全属性和**用户ID访问权限可单独配置**。
4. 地址检查：若传输的起始地址落入一个使能的区间，地址检查就通过。若传输的起始地址没有落入任何一个使能的区间，地址检查就fail。
5. 安全检查：若传输的起始地址落入一个使能的区间，就按照该区间的安全属性进行检查：
  - 区间是读安全区：标识为安全的读传输通过
  - 区间是读非安全区：标识为安全/非安全的读传输均可通过
  - 区间是写安全区：标识为安全的写传输通过
  - 区间是写非安全区：标识为安全/非安全的写传输均可通过
6. userid访问权限检查：

- 所有die的master可以通过4bit的userid，最多分成16组（组0~组15）。
- 每个区间都有16bit读权限寄存器。**每一个bit对应一个master分组**，共对应16个master分组。Bit值为1时，表示对应的master组对该区间有读权限。Bit值为0时，表示对应的master组对该区间没有读权限。例如bit[0]对应maser组0，bit[1]对应master组1，依次类推。
- 每个区间都有16bit写权限寄存器。**每一个bit对应一个master分组**，共对应16个master分组。Bit值为1时，表示对应的master组对该区间有写权限。Bit值为0时，表示对应的master组对该区间没有写权限。例如bit[0]对应maser组0，bit[1]对应master组1，依次类推。

如下图示例所示：假设有一笔userid等于13（4'b1101）的传输经过firewall，若该传输的地址落入区间0，因为区间0的读权限寄存器的bit[13]为1，所以读传输可以通过。因为区间0写权限寄存器的bit[13]为1，所以写传输可以通过。若该传输的地址落入区间1，因为区间1读权限寄存器的bit[13]为1，所以读传输可以通过。因为区间1写权限寄存器的bit[13]为0，所以写传输不可以通过。



图：用户ID访问权限示例

## 10.3 Firewall 访问控制

- 启动阶段，管理子系统功耗核启动固件经过验签后，初始化 Firewall 配置。
- 管理子系统功耗C908，可按需设置 Firewall 配置。

- 当LCS 已经进入USER，firewall 根据eHSM送出的firewall\_en信号加锁，需要鉴权才能修改 Firewall 配置
- 其他 LCS 状态默认C908可以修改firewall 配置。

### 10.3.1 Firewall 配置途径

- 所有 Firewall 配置操作必须通过管理子系统的C908 完成，使用运维核。
- Die 0和Die 1 分别使用各自的管理子系统C908进行firewall 配置
- 其他子系统的 Master 均无法直接访问或修改firewall 配置

### 10.3.2 Firewall lock 机制

针对firewall 配置k加锁，LCS+Firewall Lock Register联合控制，LCS 控制参考8.3.1.2节  
Firewall 的lock 由 eHSM进行管控



### 10.3.3 Firewall error 机制

| 类型  | Firewall 动作                                                                                                       |
|-----|-------------------------------------------------------------------------------------------------------------------|
| 合法读 | 透传                                                                                                                |
| 非法读 | <ul style="list-style-type: none"><li>• 根据error上报标识选择是否上报Response error，这一版不报 error</li><li>• 禁止访问slave</li></ul> |
| 合法写 | 透传                                                                                                                |
| 非法写 | <ul style="list-style-type: none"><li>• 根据error上报标识选择是否上报Response error，这一版不报 error</li></ul>                     |

- 禁止访问slave

## 10.3.4 Firewall 配置

### Firewall需求

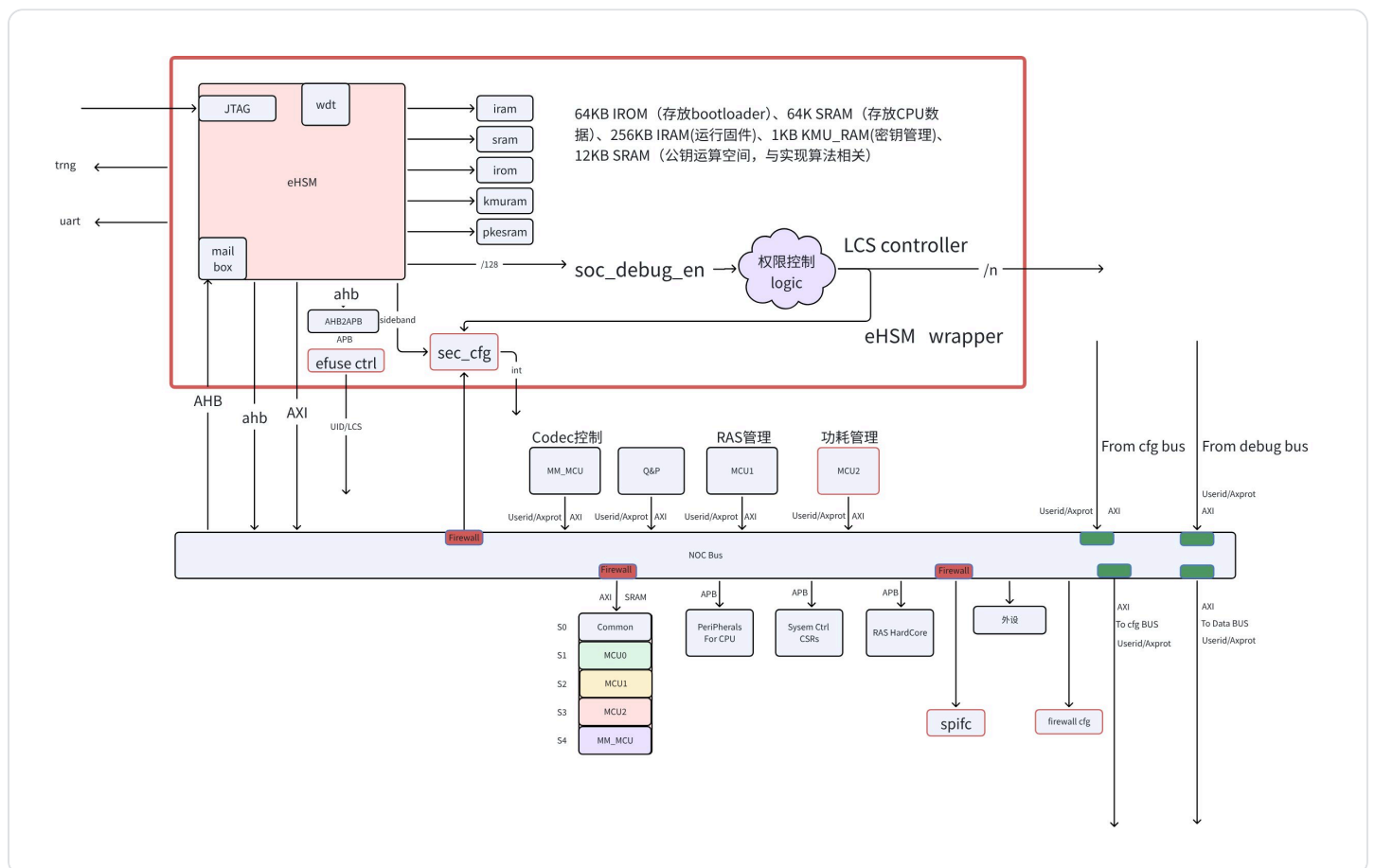
管理子系统：

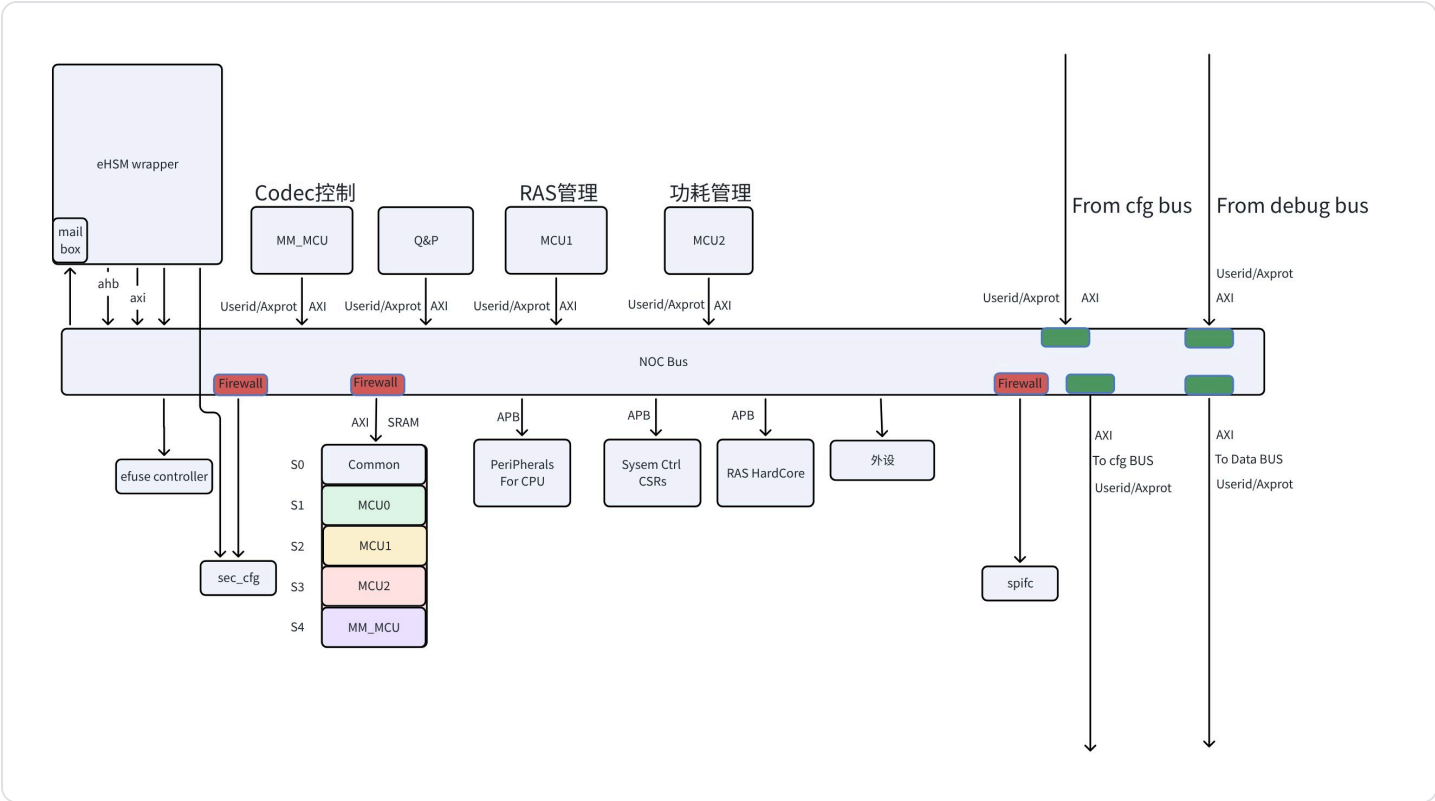
在启动过程中对管理子系统的固件做安全校验后，将其搬运到管理子系统sram中存储，为了防止被Host恶意攻击以及CPU之间的相互渗透，需要考虑：

- Sram firewall：在子系统内部的sram入口设置firewall隔离机制
- Spifc firewall
- sec\_cfg firewall

### NGU800P总线connectivity

管理子系统的firewall 位置：





1. 管理子系统firewall:

- Firewall 管控SRAM的访问
  - 将SRAM划分为5个空间，MCU0/1/2/mm\_mcu 分别只能访问自己对应的sram空间。
  - Sram common可以开放给cfg bus
- firewall管控sec\_cfg的访问
  - sec\_cfg 默认只能由功耗核MCU2 控制，其他master 可读sec\_cfg
  - firewall管控sec\_cfg的访问

NOC 访问控制

- eHSM 的mailbox ahb接口，只能由功耗核MCU2访问
- 安全efuse 只能由eHSM访问
- Firewall cfg 只能由管理子系统功耗核配置

Firewall 配置

1. sram Firewall 配置:

| Sram 分区 | 地址范围 | 寄存器配置 | 寄存器配置描述 |
|---------|------|-------|---------|
|---------|------|-------|---------|

|          |                           |                                         |                                                                                                                                                  |
|----------|---------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Region 1 | 0x0000~0x0FFF<br>(增加地址位宽) | sram_firewall_cfg<br>默认值: 0x 0000 0003  | 每2bit对应1个Master ID的读/写权限(低2bit有效)。默认值 0x00000003 表示 MCU1 可读可写 ( 0x3 ), 其余ID无权限。<br><br>0b0:不可读不可写<br><br>0b1: WO<br><br>0b10: RO<br><br>0b11: WR |
| Region 2 | 0x1000~0x1FFF             | sram_firewall_cfg<br>默认值: 0x 0000 0030  | 每4bit对应1个Master ID的读/写权限(低2bit有效)。默认值 0x000000 30 表示 MCU2 可读可写 ( 0x3 ), 其余ID无权限。                                                                 |
| Region 3 | 0x2000~0x2FFF             | sram_firewall_cfg<br>默认值: 0x 0000 00300 | 每4bit对应1个Master ID的读/写权限(低2bit有效)。默认值 0x000000 300 表示 MCU3 可读可写 ( 0x3 ), 其余ID无权限。                                                                |
| Region 4 | 0x3000~0x3FFF             | sram_firewall_cfg<br>默认值: 0x 0000 03000 | 每4bit对应1个Master ID的读/写权限(低2bit有效)。默认值 0x000003000 表示 MCU4可读可写 ( 0x3 ), 其余ID无权限。                                                                  |
| Region 5 | 0x4000~0x4FFF             | sram_firewall_cfg<br>默认值: 0x 3333 3333  | 所有master id 都可以读写访问                                                                                                                              |

2. sec cfg firewall 配置：

|         |                                       |                                                                                                                                                 |
|---------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| slave   | 寄存器配置                                 | 寄存器配置描述                                                                                                                                         |
| sec_cfg | sec_firewall_cfg<br>默认值: 0x 0000 0030 | 每4bit对应1个Master ID的读/写权限(低2bit有效)。默认值 0x00000030 表示 MCU2可读可写 ( 0x3 ), 其余ID无权限。<br><br>0b0:不可读不可写<br><br>0b1: WO<br><br>0b10: RO<br><br>0b11: WR |

3. spifc firewall 配置：

|       |                    |                                                              |
|-------|--------------------|--------------------------------------------------------------|
| slave | 寄存器配置              | 寄存器配置描述                                                      |
| spifc | spifc_firewall_cfg | 每4bit对应1个Master ID的读/写权限(低2bit有效)。默认值 0x00000030 表示 MCU2可读可写 |

|  |                  |                                                                                    |
|--|------------------|------------------------------------------------------------------------------------|
|  | 默认值：0x 0000 0030 | ( 0x3 ), 其余ID无权限。<br><br>0b0:不可读不可写<br><br>0b1: WO<br><br>0b10: RO<br><br>0b11: WR |
|--|------------------|------------------------------------------------------------------------------------|

Firewall 配置寄存器硬件安全逻辑：

firewall的寄存器每个master id配置4bit，有效位为2bit

- 如果读到的 4bit master 编码为 4'h0 ~ 4'h3 → 合法master id
- 如果读到的 4bit master 编码为 4'h4 ~ 4'hF → 判定为 4'h0 （拒绝访问），无WR权限。

Master id 硬件安全逻辑：

- 如果 master\_id≠reserved（合法 ID），正常进入firewall控制阶段。
- 如果 master\_id=reserved（非法 ID），拒绝访问，无WR权限。

其他：

1. firewall的配置由功耗核C908统一管理
2. 除管理子系统外其他地方无firewall
3. ucie在通路上是内部通路，跟随pcie的通路，不做特别限制
4. PCIE：
  - 敏感信息，软件限制无通路访问。敏感信息：efuse/固件运行rtos sram/运行linux dram/pcie remap（IATU/apt remap）
  - PCIE debug 通路受LCS管控，LCS=USER，限制PCIE debug通路的访问，默认通路不通，鉴权可打开

## 11. 认证规划

NGU800 安全设计满足国密一级设计要求。eHSM硬件设计已满足。

NGU800 安全设计遵循《人工智能加速芯片安全功能技术规范》基础级。当前feature已遵循。

## 12. 遗留问题（Open Issues）

